



UNIVERSITÀ DI TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer Science

FINAL DISSERTATION

PERFORMANCE ANALYSIS OF THE MPEG-DASH PROTOCOL FOR VIDEO STREAMING

In-depth study on simulated and real environments

Supervisor
Prof. Paolo Casari

Student
Elia Gatti

Academic year 2022/2023

Acknowledgements

I miei più profondi e sentiti ringraziamenti vanno ai miei genitori. Il vostro amore incrollabile, la fiducia nelle mie capacità e l'instancabile sostegno mi hanno sostenuto lungo questo percorso impegnativo. Sono eternamente grato per i sacrifici che avete compiuto e per il modo in cui mi avete incoraggiato in ogni momento. Desidero inoltre esprimere il mio sincero apprezzamento a tutti coloro che hanno offerto incoraggiamento, ascolto o aiuto lungo il percorso. Il vostro sostegno ha significato molto per me. Questo risultato non sarebbe stato possibile senza la gentilezza e la generosità di coloro che mi circondano.

My parents deserve my most profound and heartfelt thanks. Your unwavering love, belief in my abilities, and tireless support have sustained me throughout this demanding journey. I am eternally grateful for the sacrifices you have made and the ways you have encouraged me every step of the way. I would also like to extend my sincere appreciation to everyone who has offered encouragement, a listening ear, or a helping hand along the way. Your support has meant the world to me. This accomplishment would not have been possible without the kindness and generosity of those around me.

Contents

Abstract	3
1 Introduction	4
1.1 Problem and Motivation	4
1.2 Result and Contribution	4
1.3 Thesis Structure	5
2 Background	6
2.1 Video Streaming	6
2.1.1 MPEG-DASH (Dynamic Adaptive Streaming over HTTP)	6
2.1.2 HLS (HTTP Live Streaming)	6
2.1.3 MPEG-DASH and HLS: A Comparison	7
2.2 Video Encoding and Compression for MPEG-DASH Streaming	7
2.2.1 Video Encoding and Compression	7
2.2.2 MPEG-DASH, Video Encoding, and the Importance of Tailored Preparation	8
3 Methodology	9
3.1 Selection of the testing tools	9
3.1.1 Video Processing	9
3.1.2 Network Emulation	10
3.1.3 Server and Client Tools	11
3.2 Initial Setup	12
3.2.1 Selection and Encoding of Video	12
3.2.2 OpenDaylight Setup	14
3.2.3 Mininet Setup	14
3.2.4 Server Setup	15
3.2.5 Client-Side Setup for MPEG-DASH Analysis	15
4 Testing	16
4.1 Testing Official Structure	16
4.1.1 First Scenario (Mininet)	16
4.1.2 Second Scenario (Mininet with Traffic)	17
4.1.3 Third Scenario (University)	18
4.1.4 Fourth Scenario (AWS)	19
5 Analysis and Results	20
5.1 Selected Metrics	20
5.2 Analysis of Mininet	21
5.2.1 Average Dropped Frames	21
5.2.2 Average Startup Delay	22
5.2.3 Average Resolution Changes	22
5.2.4 Average Resolution Request Changes	22
5.2.5 Correlation between Download Rate and Buffer Length	22
5.2.6 Average Bandwidth Utilization % and Buffer Length	23

5.2.7	Correlation between Dropped Frames and Buffer Length	24
5.2.8	Correlation between Dropped Frames and Download Rate	24
5.2.9	Average Dropped Frames Percentage and Avg Resolution Changes	24
5.2.10	Average Dropped Frames Percentage and Rebuffering Events	25
5.2.11	Average Resolution Changes and Rebuffering Events	25
5.3	Analysis of Mininet with Traffic	26
5.3.1	Low Delay (Zero loss)	26
5.3.2	High Delay (Zero loss)	27
5.3.3	Low Delay (low loss)	28
5.3.4	High Delay (low loss)	29
5.4	Analysis of Impact Made by Geographical Distance	30
5.5	Result and Future Research	32

Bibliography	38
---------------------	-----------

Abstract

In the era of seamless internet access, video streaming has become an integral part of our daily lives, with a significant portion of internet users engaging with video content on a weekly basis. However, the delivery of high-quality video content over the Internet remains a challenge due to the diversity of network connections and the need for adaptive streaming technologies. This thesis addresses the critical issue of video streaming performance under varying network conditions, focusing on the MPEG-DASH protocol, a widely adopted adaptive bitrate streaming (ABR) solution. The motivation behind this research is to contribute to the understanding of MPEG-DASH behaviour, particularly in environments with unstable or limited network resources.

The problem undertaken in this thesis is the analysis of the MPEG-DASH protocol's performance in simulated and real-world network scenarios, with a particular emphasis on how the protocol adapts to variations in bandwidth, latency, and packet loss. These factors are crucial in determining the quality of video streaming and can significantly impact the user experience. The study aims to provide insights into the behavior of MPEG-DASH in different network conditions, which is essential for service providers to optimize their offer and ensure a superior viewing experience for users.

To tackle this problem, a comprehensive methodology was adopted, combining both simulated and real-world scenarios. The research utilized a variety of tools and technologies, including video processing libraries such as x264 and GPAC (MP4Box), network emulation with Mininet and OpenDaylight, and server and client tools implemented in Node.js and DASH.js. The methodology also involved the creation of a custom server setup tailored for MPEG-DASH streaming and a client-side implementation for real-time performance monitoring and data logging. The tests were conducted in a controlled environment using Mininet, a network emulator, and in real-world scenarios involving a university server and an AWS server, to compare the impact of geographical distance and realistic trans-oceanic lines on video streaming performance.

The results achieved in this thesis reveal significant insights into the behavior of MPEG-DASH under various network conditions. The analysis of such metrics as download rate, buffer length, bitrate, resolution, and dropped frames showed that the protocol dynamically adjusts to network conditions, with notable correlations between these metrics. The introduction of traffic in the network environment led to changes in buffer length and resolution changes, highlighting the protocol's ability to adapt to competing network demands. The impact of geographical distance was also examined, with minimal differences observed between tests conducted with a university server and an AWS server, suggesting that the protocol performs well.

The personal contribution of this thesis lies in the development of a detailed methodology for testing MPEG-DASH performance, the implementation of a custom server and client setup for data collection and analysis, and the interpretation of the results. The findings contribute to the broader understanding of MPEG-DASH's behavior in different network scenarios.

1 Introduction

The 2023 Global Web Index (GWI) study highlights the ever-increasing dominance of online video, demonstrating that almost 93% of global internet users engage with video content weekly [4]. Further data illustrates how platforms like Netflix and Disney+ command a significant portion of television viewing for working-age users [4]. This popularity of streaming video underscores its integration into our daily lives for purposes ranging from entertainment (music, sports, comedy) to education, product reviews, and advertising.

This growth aligns with worldwide improvements in connection speeds. Reports substantial increases in both fixed and mobile download/upload speeds with reduced latency [4]. Improved connectivity enables higher-quality online video experiences. However, challenges are still prevalent in regions with less developed infrastructure, where bandwidth limitations and packet loss hinder seamless video streaming [4]. Mobile connectivity, although improving, also remains subject to coverage fluctuations.

This variety of network conditions presents a challenge for video streaming services who want to cater to viewers worldwide. It becomes essential to deliver high-quality experiences to those with fast connections while still accommodating users hampered by lower speeds. Adaptive Bitrate Streaming (ABR) plays a crucial role by seamlessly adjusting video quality based on network conditions, device capabilities, and screen size, aiming to prevent frustrating pauses and interruptions. Two significant ABR protocols have emerged: Dynamic Adaptive Streaming over HTTP (DASH) and HTTP Live Streaming (HLS).

HLS is closely tied to the Apple ecosystem, whereas DASH stands out as an open, more universal standard. Both protocols are commonly used and compatible with various video formats, encryption, and Digital Rights Management (DRM) systems [15]. DASH, designed by the MPEG group, was established as a standard in 2011 and subsequently updated throughout the years. Its development benefits from contributions by industry giants like Microsoft, Netflix, Google, Samsung, and more [6].

Due to its open source nature, DASH is the preferred choice for many major streaming platforms including Youtube, Netflix, Disney+ and Hulu [21]. Companies like these rely on DASH to dynamically adjust video quality for individual devices and connection speeds, optimizing the viewing experience.

1.1 Problem and Motivation

This thesis delves into the complexity of the MPEG-DASH protocol, focusing on its performance within both simulated and real-world network settings. The central concern is the dynamic adaptation of the protocol to fluctuating parameters such as bandwidth availability, latency, and packet loss rates. These variables are pivotal in shaping the quality of video streaming experiences and can profoundly influence user satisfaction. By scrutinizing the protocol's responsiveness to these network conditions, this study seeks to shed light on some details of its behavior.

1.2 Result and Contribution

The empirical evidence collected in this thesis offers a deep dive into the operational characteristics of MPEG-DASH under a spectrum of network circumstances. The evaluation of key performance indicators, including download speed, buffer occupancy duration, bitrate, actual resolution, and frame drop rates, demonstrates the protocol's capacity to respond adaptively to shifting network parameters. As external network traffic was introduced, it triggered alterations in buffer lengths and resolution adjustments, underscoring the protocol's resilience in the face of concurrent network demands. Furthermore, the comparison of test results between a local university server and a cloud-based AWS server indicated negligible disparities, implying that MPEG-DASH maintains consistent performance across a variety of network distances.

The unique value proposition of this thesis is found in the meticulous methodology employed for

assessing MPEG-DASH performance, the establishment of a server and client configuration for data acquisition and analysis, and the articulation of the results.

1.3 Thesis Structure

In Chapter 2, we provide the concepts of Video Streaming, introducing MPEG-DASH, and Video encoding. Chapter 3 details the methodology adopted for testing MPEG-DASH performance, including the tools and technologies used, the creation of a custom server and client setup for the testing scenarios in simulated and real-world network environments. Chapter 4 focuses on the testing process itself, explaining the rules followed to achieve the final design and the steps necessary to set up the testing for each case. Finally, Chapter 5 presents the results of the testing, analyzing the performance of MPEG-DASH under different network conditions and the impact of geographical distance on video streaming quality.

2 Background

In this chapter, we provide the basics of Video Streaming (Section 2.1) and Video Encoding and Compression (Section 2.2).

2.1 Video Streaming

Video streaming is a critical technology that enables the delivery of video content over the Internet to viewers in real time. Unlike traditional video distribution methods, which require users to download large video files before watching, video streaming allows users to begin playing a video almost instantly after accessing the content. This is achieved through the use of various streaming protocols and technologies.

Dynamic Adaptive Streaming over HTTP (DASH) is a cutting-edge methodology for delivering multimedia content over the Internet, ensuring a seamless viewing experience by dynamically adjusting the video stream quality based on the viewer's network conditions and device capabilities. This adaptive approach is facilitated through Adaptive Bitrate Streaming (ABR), a technique that optimizes the streaming experience.

2.1.1 MPEG-DASH (Dynamic Adaptive Streaming over HTTP)

MPEG-DASH (Dynamic Adaptive Streaming over HTTP) is a sophisticated video streaming technology that enables smooth, high-quality video playback across a wide range of Internet connections and devices. At its core, it works by breaking down a video into smaller, independent segments. A special file called the Media Presentation Description (MPD) acts as a roadmap, describing the various segments, their different quality levels (bitrates), and other necessary details such as audio tracks or subtitles [15].

The "adaptive" part of MPEG-DASH lies in the client-side player software and its ability to analyze the viewer's network speed, device capabilities, and how much video data is currently stored (buffered). Based on this analysis, the player continuously selects the optimal segment to download next, ensuring that the highest possible quality video is delivered without causing annoying playback pauses or stuttering [15].

Since MPEG-DASH uses standard HTTP for delivery, just like regular web content, it benefits from existing web infrastructure. This means video is served in a way that is both very scalable and less likely to cause issues with firewalls. Moreover, standard web caching systems can store and distribute MPEG-DASH segments closer to the user, increasing performance even further [15].

Beyond core concepts, MPEG-DASH has features to support Digital Rights Management (DRM) for protecting paid content, as well as specialized techniques for minimizing delays in live video streaming scenarios. Through careful standardization, the vast majority of modern streaming devices and web browsers offer seamless MPEG-DASH compatibility [15].

In short, MPEG-DASH's design allows a single video source to serve everyone - from those on high-speed connections with big screens to those with spotty mobile networks on smaller devices. This resilience and reach are among the reasons why MPEG-DASH has become a driving force in the world of online video streaming.

2.1.2 HLS (HTTP Live Streaming)

HTTP Live Streaming (HLS) is a video streaming protocol developed by Apple. Like MPEG-DASH, it functions by segmenting a video into smaller chunks, but specifically uses the .ts (Transport Stream) file format. The protocol then creates an .m3u8 index file that acts as a playlist, telling the video player where to find the segments it needs. HLS is widely supported by Apple devices, making it a popular choice for delivering video content to iOS, iPadOS, and macOS users. It offers good adaptability and can also take advantage of standard web infrastructure for reliable content delivery. Additionally, HLS

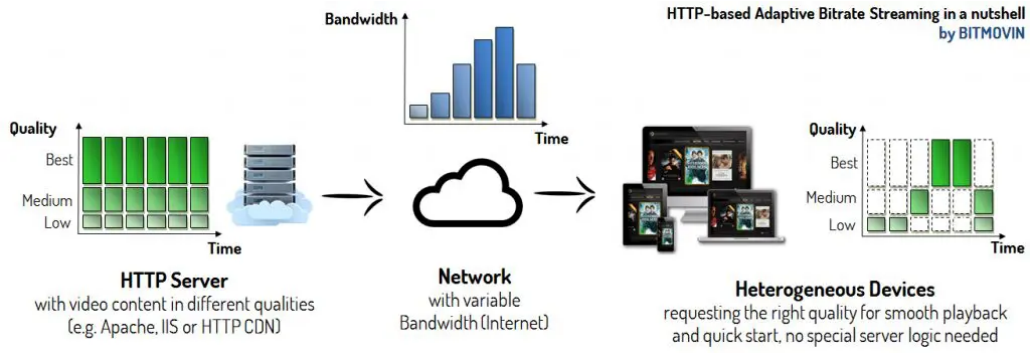


Figure 2.1: MPEG-DASH workflow

includes optimizations for handling live video streams with minimal delay. [12]

2.1.3 MPEG-DASH and HLS: A Comparison

MPEG-DASH and HLS are the main competitors in the video streaming landscape. Both are powerful adaptive bitrate streaming protocols. Their core functionality aligns, but key distinctions exist:

- **Standard:** HLS is a proprietary Apple protocol, whereas MPEG-DASH is an open-source international standard. This grants MPEG-DASH wider industry support and freedom from vendor dependencies. [11]
- **Compatibility:** HLS boasts native compatibility within the Apple ecosystem. MPEG-DASH, however, offers broader cross-device and cross-browser support due to its independence from vendor-specific implementations. [11]
- **Codec Flexibility:** Both protocols accommodate common codecs like H.264, but MPEG-DASH's open nature allows it to more readily integrate newer, potentially more efficient codecs as they emerge. [11]

DASH and HLS are both excellent choices for adaptive bitrate streaming (ABR). They offer similar picture quality, support HDR, and provide reliable, stable viewing experiences [11].

Rationale for Thesis Focus

Although HLS has wider adoption, this thesis focuses on MPEG-DASH. Its open-standard status enables in-depth examination of the protocol's internal mechanisms, a significant advantage for research purposes compared to proprietary technologies.

2.2 Video Encoding and Compression for MPEG-DASH Streaming

Video encoding and compression are fundamental aspects of video streaming, particularly for formats like MPEG-DASH. These processes transform raw video data into a format suitable for transmission over the Internet, ensuring compatibility with various devices and network conditions.

2.2.1 Video Encoding and Compression

Video encoding is the process of converting raw video data into a digital format that can be stored or transmitted efficiently [20]. This involves compressing the video data to reduce its size, which is crucial for streaming applications where bandwidth is often limited. Compression techniques aim to maintain the perceived quality of the video while reducing the amount of data required to represent it [20]. There are numerous video encoding standards and formats, each designed to balance quality, compression efficiency, and computational complexity. Some of the most widely used standards include:

- **H.264/AVC:** Advanced Video Coding (AVC) is a popular choice for video encoding due to its high compression efficiency and wide adoption [7].
- **H.265/HEVC:** High Efficiency Video Coding (HEVC) is the successor to AVC and offers even greater compression efficiency [7].

- **VP9:** Developed by Google, VP9 is another open-source video codec that competes with H.264 and H.265 [7].

2.2.2 MPEG-DASH, Video Encoding, and the Importance of Tailored Preparation

MPEG-DASH is a flexible video streaming protocol that leverages existing encoding standards like H.264 or H.265 to deliver segmented media files. Since each segment is individually encoded and compressed, MPEG-DASH enables dynamic adaptation by allowing clients to seamlessly switch between different quality representations based on network conditions [15].

However, optimizing video streaming with MPEG-DASH requires more than selecting a codec and quality level. Careful video preparation prior to DASH segmentation significantly improves the overall viewing experience. Here are some key factors to consider:

- **Two-Pass Encoding:** When dealing with content prepared in advance (not live), two-pass encoding provides significant advantages. By analyzing the video twice, the encoder gathers more information and optimizes the file size while preserving visual quality [14].
- **Constant vs. Variable Bitrate:** Constant bitrate (CBR) prioritizes predictable file sizes, whereas variable bitrate (VBR) prioritizes visual quality by dynamically adjusting the bitrate based on scene complexity. Generally, VBR results in better-looking video within a specified average bitrate [14].
- **Presets:** Encoding tools often come equipped with presets (such as "YouTube 1080p") that strike a balance between ease of use and granular control. Presets are optimized for common video streaming scenarios [14].

3 Methodology

In this chapter, we discuss the Selection of the Testing Tools (Section 3.1) and the Initial Setup (Section 3.2).

3.1 Selection of the testing tools

To address the complex challenges posed by the diverse landscape of network connections and the critical role of Adaptive Bitrate Streaming (ABR) in delivering high-quality content, this thesis adopts a comprehensive methodology. The focus is on the in-depth analysis of the MPEG-DASH protocol under varying network conditions, combining both simulated and real-world scenarios. Testing primarily occurred on my personal computer, equipped with an Intel Core i5-8300h @ 2.30GHz, boasting 4 physical cores and 8 threads, 16GB of DDR4 RAM in dual-channel mode clocked at 2667 MHz, and a GeForce GTX-1050 MaxQ Mobile GPU produced by NVIDIA. The operating system installed is POP!_OS 22.04. All scripts mentioned in this methodology are accessible in my personal GitHub Repository [18]. Additionally, for University servers, a computer with an 8-core CPU and single-channel 16GB of RAM was used. AWS machine specifications included an Intel Xeon CPU E5-2686 v4 @ 2.30GHz with 8 cores and dual-channel 32GB of RAM. This section of the chapter will focus on what tools and the motives behind why have been selected in order to conduct the testing of MPEG-DASH.

3.1.1 Video Processing

x264

x264 is a powerful and versatile open-source video encoding tool specializing in the widely-used H.264/MPEG-4 AVC format, ideal for compressing high-definition video [22]. It is favored for its ability to produce excellent quality video files efficiently. The heart of x264 lies in its focus on H264/MPEG-4 AVC, ensuring your videos are compatible with the multitude of devices and platforms that support this standard [22].

x264 excels by providing control over how videos are encoded. It makes it possible to prioritize consistent visual quality with settings such as Constant Rate Factor (CRF), even at the price of slightly larger files [10]. x264 offers extensive customization options, allowing users to fine-tune the encoding process based on specific content characteristics. This adaptability ensures optimal results for diverse video types, including fast-paced action sequences, detailed animation, and static images[10]. x264's presets automatically balance compression, quality, and encoding speed, ranging from 'slow' for maximum compression to 'ultrafast' for when time is of the essence [10].

Ultimately, x264's strength is its flexibility. It provides the tools for fine-grained control over the encoding process, making it an excellent choice for users who want to precisely fine-tune video quality and file size for various types of content. I used x264 for video encoding (see section 3.2.1). It paired well with MP4Box and avoided the segmentation errors encountered with FFmpeg [14].

GPAC (MP4Box)

GPAC (Graphics and Parallel Computing Architecture) is a versatile open-source framework designed for working with a wide range of multimedia content. It emphasizes adaptability and adherence to industry standards. A core component of GPAC is MP4Box, a powerful multimedia packager known for its extensive MP4-related capabilities.

MP4Box offers users a suite of tools suitable for a variety of tasks. With it, you can convert files between different multimedia formats, split media files into smaller segments, optimize them for streaming, and extract their metadata. Furthermore, MP4Box supports encryption and decryption and lets you directly manipulate file structures. While most of its features are accessible through

command-line controls, MP4Box can also integrate into server-side tools designed for streaming workflows [8].

GPAC has undergone significant changes, specifically designed to streamline operations and increase versatility. This includes a re-architecting around a filter-based model, allowing for more seamless integration between core tools like MP4Box and MP4Client. The introduction of the GPAC application makes it easier than ever to define and execute complex media processing chains directly from the command line [9].

In short, GPAC is a robust and comprehensive multimedia framework, making it a valuable tool for video professionals, researchers, and anyone who works with media extensively. MP4Box, as a central component, is particularly helpful for sophisticated manipulation of MP4 files. I used MP4Box to convert .264 files to MP4 format and later prepare videos for MPEG-DASH streaming (detailed in section 3.2.1).

3.1.2 Network Emulation

OpenDaylight and DLUX

OpenDaylight Controller is a flexible and powerful platform for Software-Defined Networking (SDN), allowing developers to create innovative network management solutions. It can work with many different networking technologies, making it a versatile choice. OpenDaylight's DLUX (Daylight User Interface) adds a user-friendly graphical web interface to this controller. This makes interacting with and managing the network significantly easier than using command-line tools alone.

DLUX offers several key benefits, particularly in version 0.8.4. It works with OpenDaylight's Southbound Abstraction Layer (SAL) to simplify the management of network resources [2]. Users can directly configure network devices and view the entire network topology through the web interface thanks to its integration with the YANG-based MD-SAL datastore [19]. The web interface itself is well-designed and intuitive to use. Additionally, the OpenDaylight platform (and DLUX specifically) is built for extensibility, allowing you to add custom features through plugins as needed [2].

With version 0.8.4 of OpenDaylight, DLUX becomes extremely useful for making the most of the controller. Common tasks that were once command-line only, such as logging in, monitoring network statistics, and working with the MD-SAL datastore, could now be performed through the graphical interface [19].

From a personal standpoint, having a visual representation of your network's layout through DLUX is exceptionally helpful. It is much easier to verify correct configuration and streamline the testing process with this level of visual feedback.

Mininet

Mininet is an open-source tool that enables users to create and experiment with Software-Defined Networks (SDNs) directly on their computers. SDNs are networks where the decision-making logic (control plane) is separated from the devices that actually move data (data plane). This separation makes networks much more flexible and programmable [1].

Useful features of Mininet include:

- **Realistic Simulations:** On a single machine, you can set up realistic virtual networks with controllers, switches, and hosts [1]. This makes testing and prototyping SDN implementations much easier.
- **Customization:** While it comes with standard components like Open vSwitch (OVS), you can swap in your own SDN controllers and switches as needed [1].
- **OpenFlow Support:** OpenFlow is a key protocol for SDN, and Mininet works with it out of the box. This lets you experiment with how controllers influence network behavior [1].
- **Flexible Network Design:** Use Python scripts to define simple or complex network topologies, giving you full control over your test environment [13, 1].

- **Ease of Use:** Mininet provides both a command-line interface and a Python API, allowing you to choose the interaction method that works best for your task [13].
- **External Controller Integration:** You can even connect Mininet to controllers running on separate machines for more realistic simulations [13].

Mininet offers multiple installation methods to accommodate varying user requirements. For the highest degree of customization, native installation provides direct control over configuration, albeit with a more complex setup process. Alternatively, a pre-configured virtual machine offers a simplified installation experience ideal for rapid deployment. Lastly, Mininet may be installed via your operating system’s package manager for a streamlined setup process. [5].

The overall reason Mininet is so useful is that it makes SDN accessible. The user-friendly interface, native OpenFlow support, and freedom to customize virtual networks make it a great learning tool. This versatility and ability to control my network simulation made it a must have.

3.1.3 Server and Client Tools

The selection of programming languages and libraries played a crucial role in ensuring the project’s functionality and efficient data analysis. JavaScript was selected as the primary language for both the server and client-side components. This choice was motivated by JavaScript’s widespread adoption, developer accessibility, and ease of integration within the project’s scope.

The server was constructed using Node.js, playing a pivotal role in the implementation of core project functionalities.

Dash.js

The DASH Industry Forum (DASH-IF) created DASH.js to serve as a model for playing back MPEG-DASH video content within web browsers. It is designed as a JavaScript library, working alongside standard browser technologies like Media Source Extensions (MSE) and Encrypted Media Extensions (EME) [17]. DASH.js was chosen for this role due to its emphasis on adaptive bitrate streaming (ABR), which means it can automatically analyze a viewer’s internet connection and adjust the video quality on the fly, ensuring the best possible viewing experience. Additionally, since it is pure JavaScript, DASH.js can be easily integrated into webpages without the need for additional plugins, making MPEG-DASH video playback incredibly convenient for websites [16]. Finally, by working with MSE and EME, DASH.js gracefully handles the technical details of streaming playback and works seamlessly with Digital Rights Management (DRM) protection for premium content [16].

DASH.js was integrated into the client-side code, enabling both the viewing of MPEG-DASH content and the logging of relevant statistics crucial for subsequent analysis and evaluation. The decision to utilize DASH.js within the browser stemmed from limitations encountered with popular desktop video players such as VLC and MPV.

Chart.js

Chart.js, an open-source JavaScript charting library, was used to create comprehensive and customizable visualizations of project data [3]. It was a great choice for several reasons. Firstly, Chart.js supports a wide array of chart types, including line charts, bar charts, pie charts, and more, ensuring flexibility in how the data could be represented [3]. Additionally, Chart.js incorporates interactive elements like tooltips and hover effects, allowing users to engage directly with the charts and dive deeper into the data. Finally, Chart.js emphasizes visual clarity, making the resulting visualizations both attractive and easy to interpret for the end-user [3].

Using this library significantly enhanced my ability to identify patterns and understand their causes in real-time during testing. Additionally, the library’s widespread adoption provided access to tutorials and existing implementations, streamlining its integration and customization for my specific use case.

Table 3.1: List of bitrates and resolutions used

Bitrate (kbps)	Width (px)	Height (px)
12000	3840	2160
8000	2560	1440
6800	1920	1080
5000	1280	720
3500	1024	576
3000	960	540
2000	768	432
1500	640	360
800	480	270
500	320	180

3.2 Initial Setup

3.2.1 Selection and Encoding of Video

The chosen video is "Big Buck Bunny" from the Blender Foundation ¹, specifically the 4K 30fps version, considered a representative best-case scenario due to the widespread support for 4K resolution in modern televisions.

To streamline the testing process, the 10-minute original video is trimmed to 5 minutes, aiming to improve encoding times and reduce the size of the logging file while maintaining precision.

After the initial video processing, the focus shifts to determining optimal resolution and bitrate combinations to create multi-resolution and multi-bitrate representations of the same video. The x264 command is employed, running the x264 encoder, a free software library for encoding video streams into the H.264/MPEG-4 AVC format. The chosen resolutions and bitrates are depicted in 3.1

Other parameters were set as follows:

- `--output "$f.$resolution.264"` Specifies the output file name. It uses the variable `f` (the file name), appending an underscore, the resolution, and the `.264` extension for a raw H.264 stream.
- `--fps 30` Sets the frames per second of the output video to 30.
- `--preset slow` Sets the encoding preset to 'slow'. This preset determines the speed of the encoding process. A slower preset provides better compression (smaller file size for the same quality). It is generally a good default choice.
- `--bitrate $bitrate` Sets the target bitrate in kbps for this representation.
- `--vbv-maxrate $maxrate` and `--vbv-buFSIZE $bufsize` These options relate to the video buffering verifier (VBV), a model of the decoder buffer in video encoding specifications. `maxrate` is the maximum rate at which data can be taken from the buffer, and `bufsize` is the size of the buffer. A rule of thumb is to set `--vbv-maxrate` to double the bitrate and `--vbv-buFSIZE` to double `--vbv-maxrate`.
- `--keyint 120 --min-keyint 120` These options set the maximum and minimum interval between IDR-frames (keyframes) in the output video to 120 frames. `keyint` should match the desired segment length in seconds multiplied by the frame rate (4 seconds * 30 frames/seconds = 120 frames). `--min-keyint` is set to the same value to achieve constant frame length.
- `--no-scenecut` Disables adaptive I-frame decision, ensuring the encoder will not insert a keyframe when it detects a scene change. This ensures the parameters above are followed precisely.
- `--pass 1` This is for two-pass encoding. In the first pass, the encoder generates some statistics about the video used in the second pass to achieve better encoding quality. Only one pass is used; a higher value can further improve the result quality but takes longer to compute.

¹<https://download.blender.org/demo/movies/BBB/>

`--video-filter "resize:width=$width,height=$height"` Applies a filter to resize the video to the specified width and height.

`"${fe}"` The source video.

Testing revealed that specific parameter choices ensure consistent keyframe and encoding alignment across resolutions, addressing potential issues when switching resolutions. However, audio loss during this process led to excluding audio from the tests. Challenges with the H.264 encoder and h264Nvenc in previous ffmpeg commands were encountered, with the former not launching and the latter generating excessive audio files for each resolution.

The generated files had a .264 extension, making direct encoding impossible. To resolve this, the MP4Box command was used to convert the .264 files into .mp4 format. Here's a breakdown of the command:

```
MP4Box -add "${f}_${resolution}.264" -new "${f}_${resolution}.mp4"
```

Next, the converted .mp4 files are processed with another MP4Box command to create a DASH manifest:

```
MP4Box -dash 4000 -frag 4000 -rap -profile "dashavc264:live" -out "${f}.mpd" "${f}_2160.mp4" "${f}_1440.mp4" "${f}_1080.mp4" "${f}_720.mp4" "${f}_576.mp4" "${f}_540.mp4" "${f}_432.mp4" "${f}_360.mp4" "${f}_270.mp4" "${f}_180.mp4"
```

Here's what each option does:

`-dash 4000` Segments the given file into 4000ms (4 seconds) chunks.

`-frag 4000` Sets the fragmentation time in milliseconds, specifying how often the media segments should be split into fragments. Setting it to the same value as `-dash` results in one subsegment per segment.

`-rap` Indicates that the first frame of each media segment should be a random access point (RAP), i.e., a keyframe. This allows playback to begin at any point within the segment without downloading the entire segment. The segment duration may vary due to the placement of keyframes in the video, which is why the video was previously encoded with the appropriate settings.

`-profile "dashavc264:live"` Specifies the profile to be used for the DASH content.

The `dashavc264:live` profile is typically used for live streaming scenarios with H.264 encoded videos.

`-out "${f}.mpd"` Specifies the output file for the DASH manifest.

`"${f}_2160.mp4" ... "${f}_180.mp4"` These are the input video files that will be included in the DASH manifest. They are listed in descending order of resolution, starting with 2160p (4K) and going down to 180p.

In summary, this command takes a series of video files with different resolutions, creates a DASH manifest file that references these videos, and configures the manifest for live streaming with specific segment and fragment durations.

After this command is completed, no further action is required for video encoding. All of this code can be run automatically using the script called `bash.sh`.

3.2.2 OpenDaylight Setup

The OpenDaylight controller is installed on a Virtual Machine running Ubuntu Server 22.04 with specific resource allocations for optimal performance. VirtualBox is chosen as the virtualization software, and a host-only network is configured with a manual setup of IPv4 Address and Network Mask.

Java 8 is required for OpenDaylight, and the installation process involves updating the .bashrc file with the Java home path and verifying the installation. The OpenDaylight controller is then downloaded, and the service is started.

Additional features are installed to enhance functionality, including Layer 2 switching, a web-based GUI, and network topology visualization. Use the following ODL console command for feature installation:

```
feature: install odl-l2switch-all odl-dluxapps-core odl-dluxapps-topology
```

OpenDaylight is accessed done through a web browser with a specific IP address and login credentials. An important note is provided regarding the display of network topology in DLUX, requiring the use of the "pingall" command in Mininet to trigger control traffic.

Finally, in order to shut down the ODL controller either the "logout" command or "CTRL + D" are used.

3.2.3 Mininet Setup

In the Mininet setup process, version 2.3.1b4 was chosen for installation, with the Native install method selected due to its flexibility beyond Ubuntu and Fedora, and the potential for enhanced performance by directly installing on the system, allowing GPU utilization for video processing. The official Mininet download page ² offers installation instructions, with slight modifications required for distributions derived from Ubuntu or Fedora.

For handling Ubuntu-based distributions, an edit to the `mininet/util/install.sh` script was made, treating Raspbian as Debian and various Ubuntu flavors as Ubuntu. There is a pending Pull Request addressing this modification in the Mininet project.

```
# Edit mininet/util/install.sh
# Around line 86, add the following code snippet:
# Treat Raspbian as Debian
[ "$DIST" = 'Raspbian' ] && DIST='Debian'

# Treat other Ubuntu flavours as Ubuntu
UBUNTU_VARIANTS='Pop|elementary|Ubuntu|LinuxMint'
echo $DIST | egrep "$UBUNTU_VARIANTS" >/dev/null && DIST='Ubuntu'

# Limit supported distros for clarity
DISTS='Ubuntu|Debian|Fedora|RedHatEnterpriseServer|SUSE-LINUX'
if ! echo $DIST | egrep "$DISTS" >/dev/null; then
    echo "Install.sh currently only supports $DISTS."
fi
```

The network structure created in Mininet comprises a remote controller named 'c0' with IP '192.168.56.9' and port '6633'. Two Open vSwitch kernel switches ('s1', 's2') are employed, along with four hosts ('h1' - 'h4') having IPs in the '10.0.0.0/8' range. The hosts are connected to the switches, with 'h1' and 'h2' linking to 's1', 'h3' and 'h4' to 's2', and a link connecting 's1' and 's2'. The links use the TCLink class to enable traffic control parameters such as bandwidth, delay, and loss, specifically applied to the 's1'-s2' link in order to achieve a bottleneck.

The rationale behind this network setup lies in its simplicity and effectiveness, providing flexibility with multiple hosts and customizable network conditions on the inter-switch link. The choice of four hosts will be further explained in a later section 4.1.2.

²<http://mininet.org/download/>

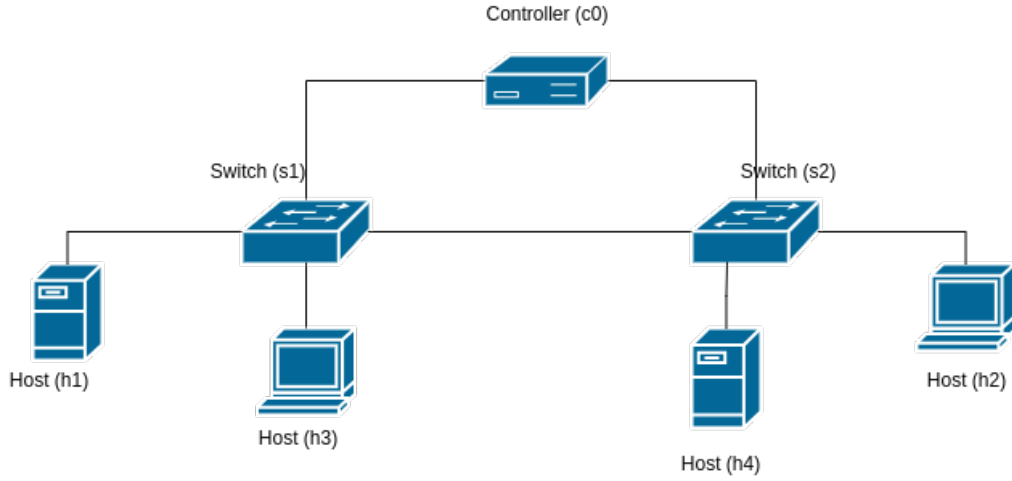


Figure 3.1: Network Structure

3.2.4 Server Setup

A Node.js server has been designed to deliver video files for MPEG-DASH streaming, focusing on performance for a video streaming thesis. Upon initialization, the server imports required modules, sets up its IP address and port, and prepares for incoming requests with features for smooth startup and shutdown.

The server handles various requests intelligently. It quickly processes favicon requests for efficiency. Upon requests to the root URL, it provides an index.html file, and it dynamically maps other URL paths to appropriate files for retrieval. The server identifies content types based on file extensions, allowing the client to accurately interpret video data.

Importantly, it supports range requests, essential for MPEG-DASH. This means the server delivers specific video segments (with a status code of 206), empowering clients to adjust video quality based on network conditions. Resource management is also considered, with the server efficiently tracking and releasing file streams as needed to ensure stability even under heavy usage.

3.2.5 Client-Side Setup for MPEG-DASH Analysis

The client-side JavaScript implementation is designed for real-time analysis of MPEG-DASH video streaming. It begins by defining parameters and creating data storage structures. The Dash.js library powers video playback, and event logging is configured for detailed tracking. Upon page load, video playback initiates alongside data capture. Chart.js enables real-time visualization of performance metrics. Throughout the session, data is continuously saved to a CSV file, which users can download for further offline analysis.

This setup emphasizes understanding MPEG-DASH performance. Real-time data reveals how network conditions and encoding choices influence the viewing experience, facilitating the identification of trends within streaming behavior. The HTML interface prioritizes user-friendliness, offering a video player, CSV download functionality, and live charts. Careful styling ensures a visually appealing and easy-to-use interface.

4 Testing

In this chapter, we explain the Testing Official Structure (Section 4.1)

4.1 Testing Official Structure

The design of the testing structure underwent constant evolution throughout the working of this thesis. The final design aims to evaluate the performance of MPEG-DASH under a broad range of network conditions, focusing on four main streaming scenarios. Among these four cases, the first two were conducted using Mininet in a simulated environment with various network parameters, while the other two were performed in a real-world streaming scenario from servers with different geographical locations.

4.1.1 First Scenario (Mininet)

The first streaming scenario aims to leverage the capabilities offered by Mininet to control network conditions. To achieve this, a set of multiple bandwidths, latencies, and packet losses was selected for a more comprehensive analysis:

- Bandwidth (Mbit/s): 50, 20, 10, 5, 3
- Latency (ms): 100, 20
- Loss (%): 2, 0

Tests in this case were conducted by applying a tuple from the superset of these three categories combined at the bottleneck link defined in the network structure (Section 3.2.3). For each tuple applied at the bottleneck link, the test was conducted a total of three times. This choice aimed to reduce potential unwanted factors that could significantly affect the final data analysis. This scenario only utilizes two out of the four hosts defined in the network structure (Section 3.2.3). The focus here is solely on how network parameters can influence MPEG-DASH streaming.

Before running this test, two main steps are required to start the experiment. The first step is to initiate the ODL controller, and the second is to locally install the files `chart.js`, `chart.umd.js.map`, `dash.all.min.js`, `dash.all.min.js.map`. The Mininet network is not connected to the internet, preventing the browser from contacting the local CDN to download the necessary files. After completing the download, the files can be added to the client folder and referenced in the `index.html` file as follows:

```
<!DOCTYPE html>
<html>
<head>
  <!-- ... -->
  <script src="../client/dash.all.min.js"></script>
  <script src="../client/chart.js"></script>
</head>
<body>
  <!-- ... -->
  <script src="../client/client.js"></script>
</body>
</html>
```

After completing these two steps, the test can be launched.

To run this test, execute the `thesis_mininet.py` file found inside the `net_conf` folder. The command run at the top-level folder (the folder `thesis` for better understanding) can be the following:

```
$ sudo -E python3 net_conf/thesis_mininet.py -bw [bandwidth] -delay [delay] -loss [loss]
```

Here is a breakdown of this command:

`sudo -E` This runs the following command with root privileges, necessary for Mininet.

`python3 net_conf/thesis_mininet.py` This runs the `thesis_mininet.py` script.

`-bw [bandwidth]` Sets the bandwidth of the link between the two switches in the network (Mbit/s).

`-delay [delay]` Sets the delay of the link between the two switches (ms).

`-loss [loss]` Sets the packet loss rate of the link between the two switches (%).

After running this command, the Mininet CLI will appear in the terminal. Type `pingall` inside it to check that each host is connected and everything is working. If there is high packet loss, some hosts may not ping each other. In such cases, rerun the command; if the issue persists, it may be due to an error in the ODL controller. In that case, a restart of the ODL controller should fix the issue.

After the initial launch and connection test, we launch `xterm` on `h1` and `h3` by typing `xterm h1 h3`. The host `h1` will be the server, so run the command `$ node server_mininet/server_mininet.js`. After launching this command, we open a browser (in this thesis, Google Chrome was used) on 'h3' using the command `$ google-chrome --no-sandbox`. This command is necessary to run Google Chrome as a root user.

Next, we navigate to the URL where the node server is hosted (`http://10.0.0.1:1337/`). By accessing this link, the server will send the correct files to the user and start the stream automatically. After the video finishes, the user can download the logged statistics by pressing the download button. This will download three CSV files containing different metrics, which will be explained in section 5.1. Also, for each test, the HAR file from the network diagnostic window found in the Developer tools is downloaded. The reason for this installation will also be explained in section 5.1.

As before, the stream was played three times by reloading the page. After finishing testing a tuple of bandwidth, delay, and loss, the Mininet experiment is closed by typing `exit` in the Mininet CLI and later executing the command `$ sudo mn -c`, which will deeply clean the processes and files generated by Mininet.

The server, client, and HTML page files used in this scenario can be found by looking inside the repository at the location `thesis/server_mininet/`. The filenames are `client.js` for the client, `server_mininet.js` for the server, and `index.html` for the HTML page.

4.1.2 Second Scenario (Mininet with Traffic)

This network scenario is similar to the first one, but with the added traffic generated between the last two hosts. In this scenario, the chosen bandwidth, delay, and loss parameters were as follows:

- Bandwidth (Mbit/s): 50, 5
- Latency (ms): 100, 20
- Loss (%): 2, 0

The testing of this network scenario aimed to observe how MPEG-DASH reacted to the addition of traffic inside the bottleneck link.

To perform the test, we first need to complete the initial setup of the ODL controller and Mininet, as explained in the chapter above. Afterward, open `xterm` on hosts `h1`, `h2`, `h3`, and `h4` by typing the command `xterm h1 h2 h3 h4`. The traffic is created using the `iperf` command by generating traffic from a host to a server. Since the links in Mininet are essentially two links, one for each direction of the connection, we need to set up the traffic to follow the original traffic generated by the video streaming. This can be achieved by having the host generate the traffic on `h2` and the server, the receiver, on `h4`. Use the following two commands, the first inside the terminal of `h4` and the second in the terminal of `h2`:

```
# On h4
$ iperf -s -u -i 1

# On h2
$ iperf -c 10.0.0.4 -u -b [traffic_bandwidth]M -t [duration]
```

The command to start `iperf` in server mode with UDP on h4 receives UDP traffic generated by host h2. The command on h2 includes several flags:

- c Specifies the server to connect to.
- u Indicates that the traffic should be UDP.
- b [traffic_bandwidth]M Sets the bandwidth to the specified value (Mbps).
- t [duration] Sets the test duration to the specified value (seconds).

The traffic used in this test was set to be 75% of the bottleneck link capability. For the chosen bandwidths of 50 Mbps and 5 Mbps, this translates to 37.5 Mbps and 3.75 Mbps, respectively. The traffic was generated after a minute of playback to allow the stream to start and adapt to the network conditions. The test duration was exactly three minutes, providing an additional minute for readaptation after the traffic had finished. This test allowed us to observe how the MPEG-DASH protocol responded to dynamic changes caused by the contention of the bottleneck link bandwidth between the video stream and the traffic. The test was conducted similarly to the previous chapter but with fewer network parameters to be tested.

4.1.3 Third Scenario (University)

This scenario is based on real-world testing, aiming to assess how MPEG-DASH performs in a favorable real-world streaming scenario, involving video streaming from a server inside the DISI department at Povo. To perform this test, the following actions need to be taken:

1. Change the `client.js` and `server.js` IP inside the code to reflect the changed IP of the machine used for testing at the University.
2. Change the `index.html` to fetch the needed files from the CDN because we now have access to the internet.

The first action can be achieved by editing and entering the IP of the machine; in this case, the server had IP 10.203.0.207, replacing where previously 10.0.0.1 was present. The second action can be done by changing the code inside `index.html`:

```
<!DOCTYPE html>
<html>
<head>
  <!-- ... -->
  <script src="https://cdn.dashjs.org/latest/dash.all.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<!-- ... -->
</html>
```

Now the necessary code will be fetched by the browser when the page is loaded. After these actions are taken, to perform the test, we can directly open Google Chrome and navigate to the URL of the server IP, in this case, `http://10.203.0.207:1337/`. After this, we can continue the testing in the same way done previously by waiting for the video to finish and collecting the data towards the end. In this case, the test was run exactly three times, collecting all the necessary data. The server, client, and HTML page files used in this scenario can be found by looking inside the repository at the location `thesis/server_over_network_uni/`. The filenames are `client.js` for the client, `server_over_network.js` for the server, and `index.html`.

4.1.4 Fourth Scenario (AWS)

This scenario is based on real-world testing, utilizing a server located in the state of Virginia (USA) in the us-east-1 datacenter. The only working method to make the connection work was through SSH tunneling by linking the localhost of my personal machine to the localhost of the other machine. The setup of the SSH tunneling was done automatically with the use of Visual Studio Code, which is the editor used to write all of the necessary code.

To run the test, we need to change the IP to localhost in both the client and server JavaScript implementations. Now, to open the stream in the browser, simply open Google Chrome and type `http://localhost:1337/`. This will open the page where the testing was conducted in the same manner as before. Also, in this case, three tests were conducted.

The server, client, and HTML page files used in this scenario can be found by looking inside the repository at the location `thesis/server_over_network_aws/`. The filenames are `client.js` for the client, `server_over_network.js` for the server, and `index.html`.

5 Analysis and Results

This chapter explores key performance metrics (Section 5.1) and analyzes them across various scenarios: Mininet without traffic (Section 5.2), Mininet with traffic (Section 5.3), and geographically distributed environments (Section 5.4). Finally, we discuss results and future research directions (Section 5.5)

5.1 Selected Metrics

To draw conclusions, it is imperative to carefully choose the metrics for monitoring, understand the reasons behind their selection, and determine the approach for monitoring them. The initial metrics to be covered involve the data logged within three generated CSV files, which include download rate, buffer length, bitrate, actual resolution, and dropped frames.

The logging of these metrics is achieved through a combination of event listeners and intervals that periodically check specific values.

- **Download Rate:** The `logDownloadRate` function logs the download rate, triggered by the `FRAGMENT_LOADING_COMPLETED` event from the Dash.js player. This event occurs when a media segment finishes loading. The download rate is calculated by dividing the total bytes of the media segment by the elapsed time since the start of the download, converted to megabits per second.
- **Buffer Length:** The buffer length is logged every second by the `eventPoller` interval, using the `getCurrentBufferLevel` method from the Dash.js player's metrics to obtain the current buffer level for the video.
- **Bitrate:** The bitrate is logged in the `logMoreData` function, triggered by the `QUALITY_CHANGE_RENDERED` event from the Dash.js player. This event happens when the video quality changes. The bitrate is retrieved from the `getBitrateInfoListFor` method of the Dash.js player.
- **Actual Resolution:** Also logged in the `logMoreData` function, the actual resolution is obtained from the `getQualityFor` method of the Dash.js player, which returns the current quality index for the video. This index is used to extract the actual resolution from the `RESOLUTIONS` array, representing the displayed resolution.
- **Dropped Frames:** The number of dropped frames is logged in the `logMoreData` function, retrieved from the `getCurrentDroppedFrames` method of the Dash.js player's metrics. It considers the sum of all dropped frames from the start of the recording, providing the total number of dropped frames.

The grouping of Download Rate, Buffer Length and Bitrate is based on their close relation and similar order of magnitude. This data is saved in the `data.csv` file. Actual Resolution is separated and saved in the `resolutionData.csv` file due to its different axis. Dropped Frames are also separated for easier plotting and analysis, considering their variable values.

In each test run, as mentioned in section 4.1.1, the HAR file of the streaming data will be downloaded. This file contains valuable information regarding network activity during video streaming, including timings for each segment downloaded. The precise data logged includes:

blocked: waiting for a network connection

connection: time to establish a network connection

send: time to send the request

wait: time to wait for the start of the response

receive: time to wait for the full response

_blocked_queueing: waiting time on the client side

This detailed data allowed for a better understanding of the timing of each video segment download. It also facilitated the analysis of:

startup delay: calculated as the time difference (in ms) from the client starting to download the MPD file to the end of the first video segment download named "dash1.m4s".

video segments resolution fetched: saved with resolution names, enabling an understanding of the resolution at which video segments were fetched.

In summary, these metrics, when analyzed in the subsequent phases, provided a deeper understanding of MPEG-DASH behavior in various network scenarios.

5.2 Analysis of Mininet

In this section, we will analyze the metrics obtained during testing inside Mininet without traffic and explore the correlations between some of them.

5.2.1 Average Dropped Frames

In this case, the analysis involved examining the correlation between the average dropped frame rate computed for the three tests for each tuple of network parameters. The heatmap displays data arranged in cases combining low and high delay (20ms and 100ms) and zero and low loss (0% and 2%). Observing the heatmap, it becomes apparent that the area corresponding to tests conducted with bandwidths of 5, 10, 'high delay and Zero loss,' and 'low delay and low loss' exhibits the highest number of dropped frames. See Figure 5.1.

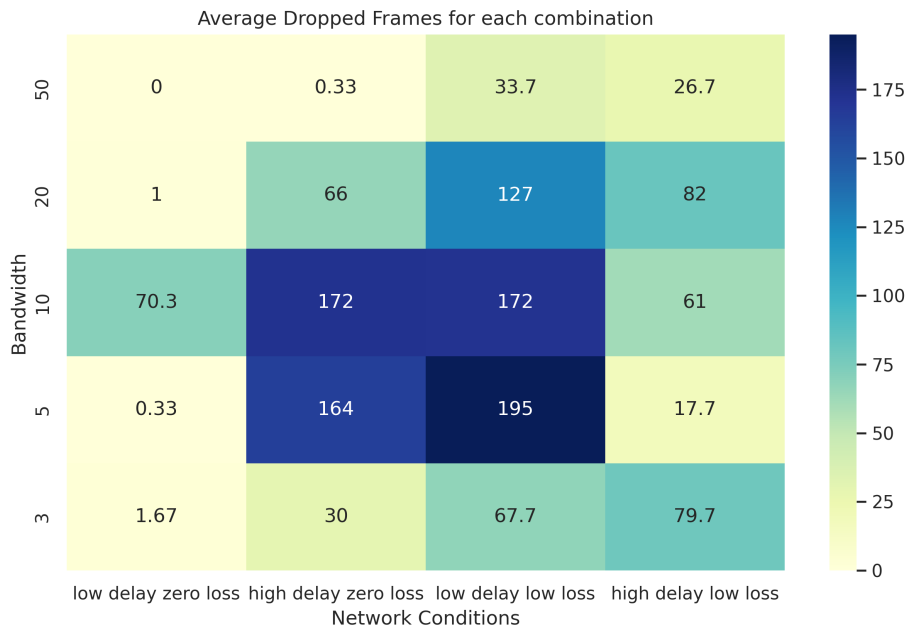


Figure 5.1: Average Dropped Frames

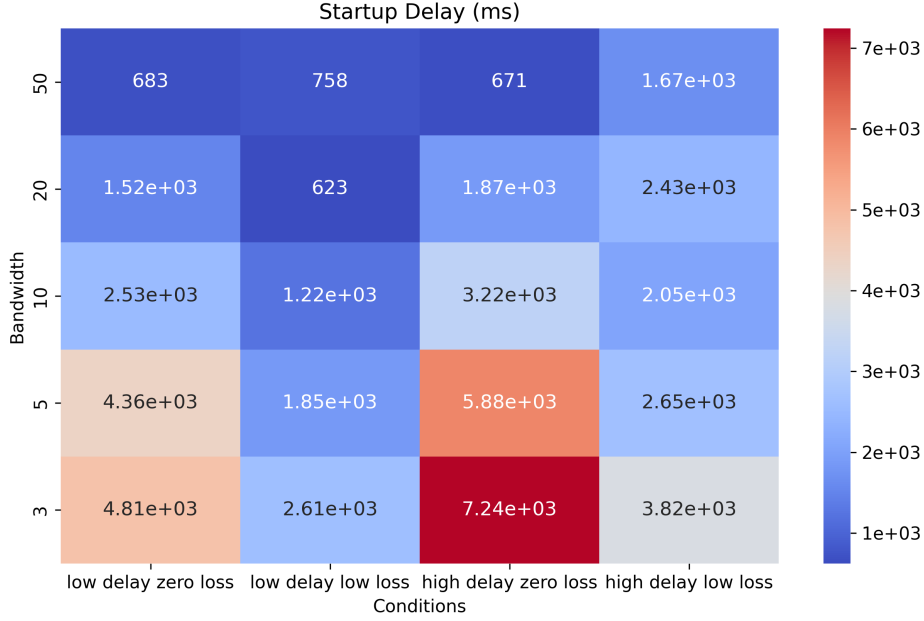


Figure 5.2: Average Startup Delay

5.2.2 Average Startup Delay

The analysis focused on how startup delay changes with varying network conditions. The heatmap illustrates that higher bandwidth correlates with generally lower startup delay. In conditions with high delay and lower values of bandwidth the gap between low and high delay increased, peaking at 7244 ms in the network condition with 3 Mbps, high delay, and Zero loss. Notably, there is a peculiar correlation between values, particularly in cases with 5 Mbps and low delay and low loss and high delay and Zero loss. Upon further investigation, it was found that the client selected the lowest resolution (180p) in the case of low delay and low loss, while in the case of high delay and Zero loss, the client often started with a higher resolution (432p), resulting in a significant delay before dropping to a lower resolution (usually 180p and occasionally 270p). Similar conditions applied to the network conditions with 3 Mbps. See Figure 5.2.

5.2.3 Average Resolution Changes

This analysis aimed to determine how frequently the resolution changed across all network conditions. The number of resolution changes did not exhibit a clear pattern, but some conclusions can be drawn. When bandwidth was at its maximum (50 Mbps), regardless of network conditions, the average resolution change was not over 1. Interestingly, the highest average of resolution changes occurred when the network conditions were 10 Mbps with low delay and Zero loss. This suggests that the client struggled to stick to one resolution and attempted an aggressive approach to maximize resolution, resulting in frequent changes. See Figure 5.3.

5.2.4 Average Resolution Request Changes

In this case, the analysis focused on the average number of times the client requested a resolution change. This data, gathered by examining all client-requested segments, tends to be higher than the visually observed actual resolution changes. This is because the client, given the opportunity, tends to request a higher or lower resolution depending on the current network condition. The graph indicates a peak in the number of requested resolution changes in conditions with high delay and Zero loss, with a maximum average of 37 requested changes. See Figure 5.4.

5.2.5 Correlation between Download Rate and Buffer Length

The analysis here involved examining the correlation between the average download rate computed for the three tests for each tuple of network parameters and the corresponding average buffer length.

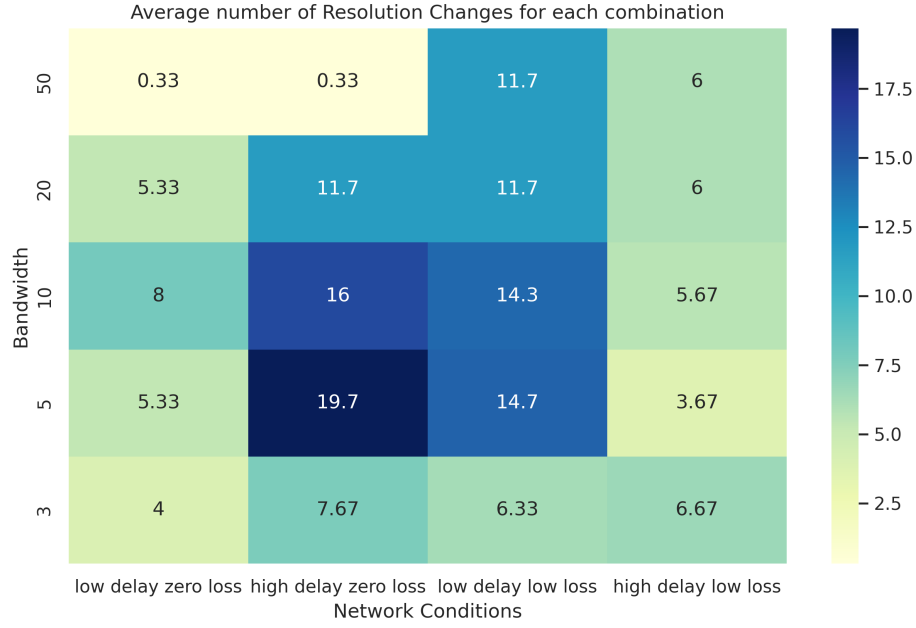


Figure 5.3: Average number of resolution changes

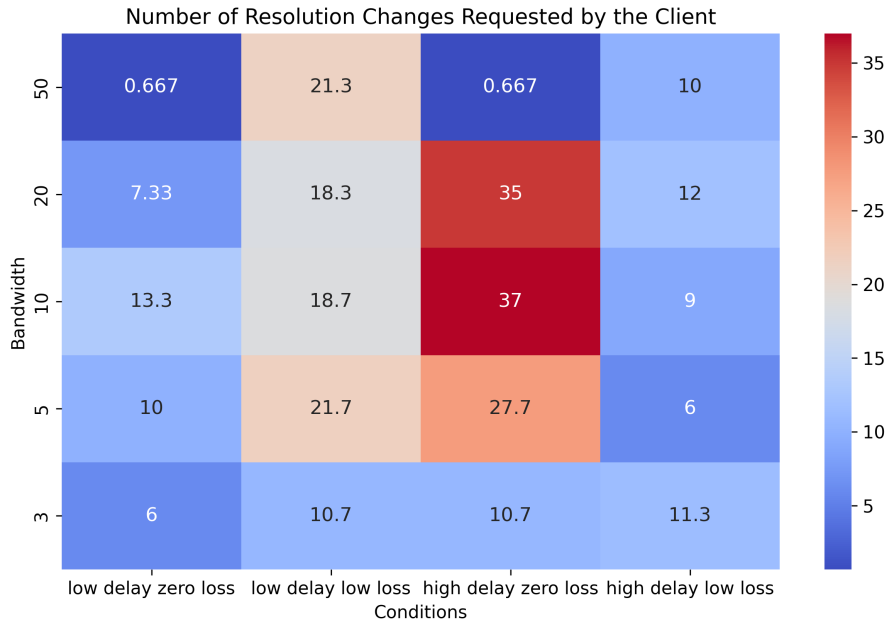


Figure 5.4: Average requested resolution changes

The graph illustrates a strong correlation between download rate and buffer health, supported by a Pearson correlation value of approximately 0.93, confirming that a higher average download rate corresponds to a longer average buffer length. See Figure 5.5.

5.2.6 Average Bandwidth Utilization % and Buffer Length

This analysis focused on how bandwidth utilization impacted buffer length. Observing the scatter plot yields several clues:

1. In tests conducted with low loss (2%), bandwidth utilization resulted in poor use, leading to an unstable buffer with limited growth.

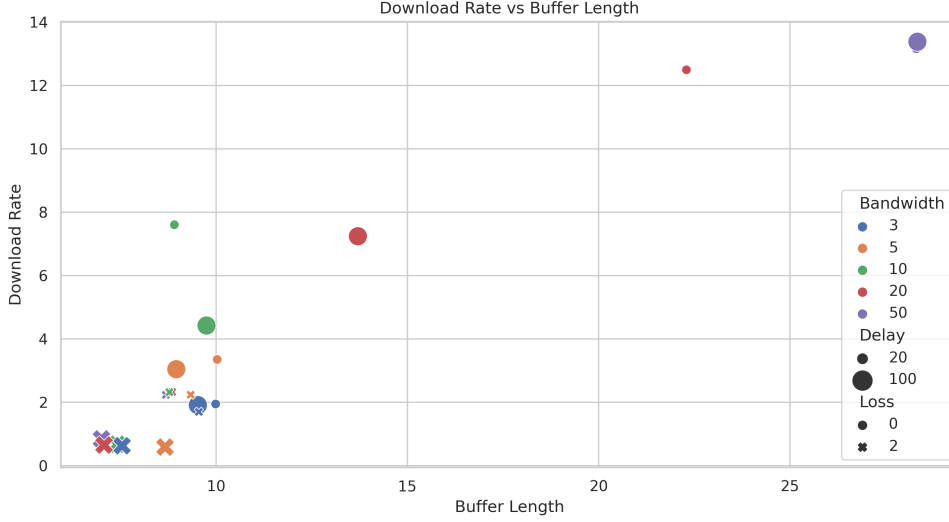


Figure 5.5: Download rate and buffer health

- Higher bandwidth utilization resulted in a lower buffer length, primarily because high bandwidth utilization occurred when the client requested a resolution maximizing available bandwidth, requiring sustained high utilization. Some bandwidth values led to different resolution choices, and when the client requested a new resolution, the buffer was usually emptied, reinforcing this result. This issue was prominent for bandwidths ranging from 3 to 10.
- The highest buffer length was achieved in tests with stable connections and low bandwidth utilization. Examining the data from a run utilizing bandwidth = 50, delay = 20, and loss = 0 (see Figure 5.6), it becomes evident that the connection was fully utilized initially to load the buffer. After this phase, the connection was used to load necessary segments periodically, ensuring a consistently full buffer.

5.2.7 Correlation between Dropped Frames and Buffer Length

This analysis aimed to identify the correlation between the average dropped frames computed from the three tests for each tuple of network parameters and the corresponding download rate. The results indicate that dropped frames are not a particularly good indicator of buffer length, supported by a Pearson correlation value of -0.43, indicating a low level of correlation. This is reflected in the scatter plot (see Figure 5.7).

5.2.8 Correlation between Dropped Frames and Download Rate

Here, the analysis examined the correlation between the average dropped frames computed from the three tests for each tuple of network parameters and the corresponding download rate. The results indicate that dropped frames are not a reliable indicator of download rate, as shown by a Pearson correlation value of -0.34. The scatter plot clearly illustrates the lack of a significant correlation (see Figure 5.8).

5.2.9 Average Dropped Frames Percentage and Avg Resolution Changes

In this analysis, the goal was to visually analyze how an increase in average dropped frames correlates with an increase in resolution changes. Looking at the figure 5.9, it is evident that a higher value of average dropped frames does not necessarily result in more resolution changes. The graph shows the highest dropped frames percentage falling halfway on the scale of resolution changes, indicating a lack of correlation.

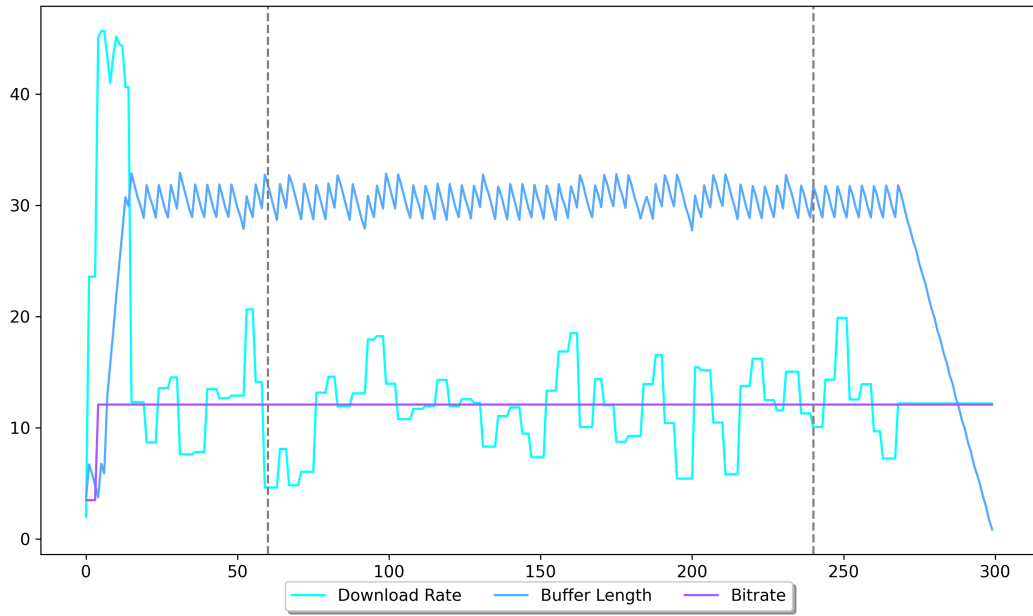


Figure 5.6: Spike in Download Rate

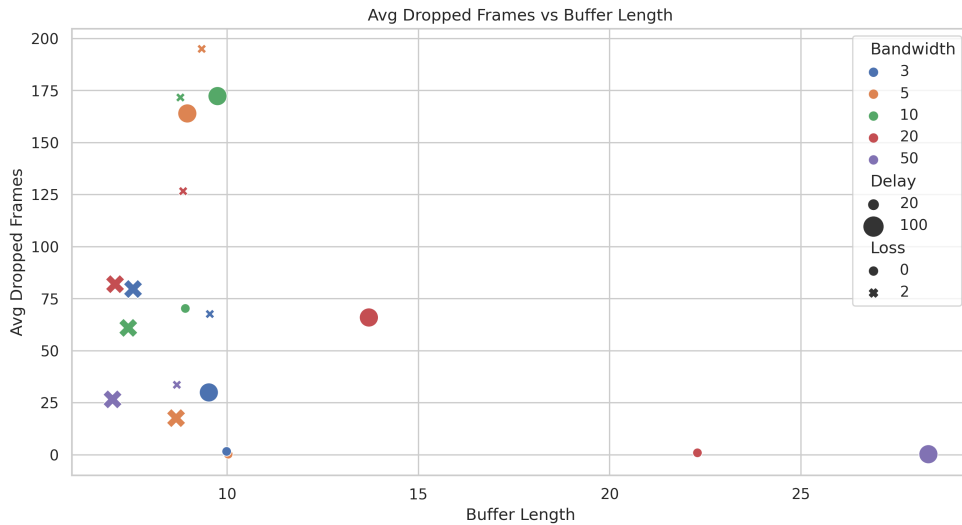


Figure 5.7: Average frames dropped and buffer health

5.2.10 Average Dropped Frames Percentage and Rebuffering Events

This analysis aimed to visually examine how average dropped frames affected average rebuffering events. Rebuffering events were calculated by counting instances where the buffer length dropped below the set threshold of 0.5 seconds. The figure 5.10 suggests that a higher average dropped frames value does not necessarily correlate with increased rebuffering events. Although all high values of rebuffering events had dropped frames, a direct correlation was not observed.

5.2.11 Average Resolution Changes and Rebuffering Events

In this analysis, the goal was to visually analyze how the average number of resolution changes correlates with average rebuffering events. The data indicates that one does not necessarily imply the other; they do not exhibit any linear correlation. Although, network conditions with high delay

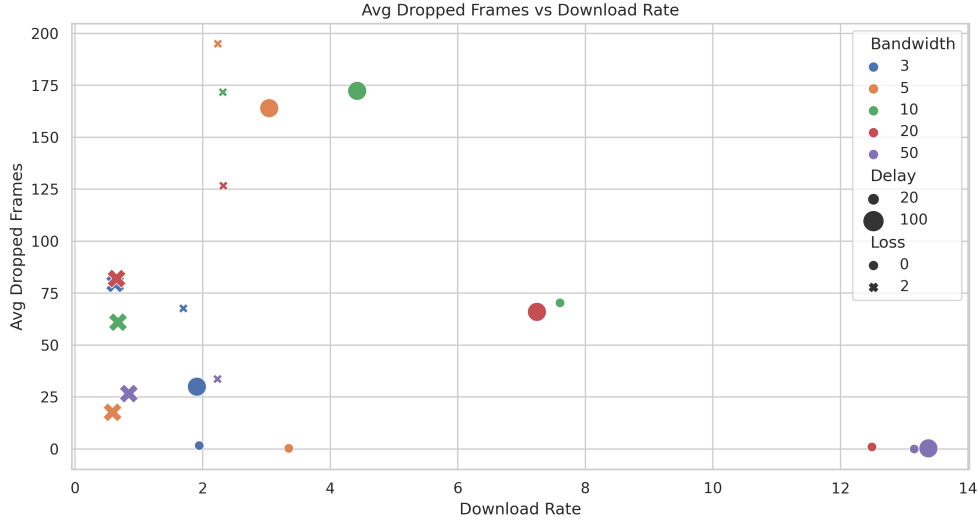


Figure 5.8: Average frames dropped and download rate

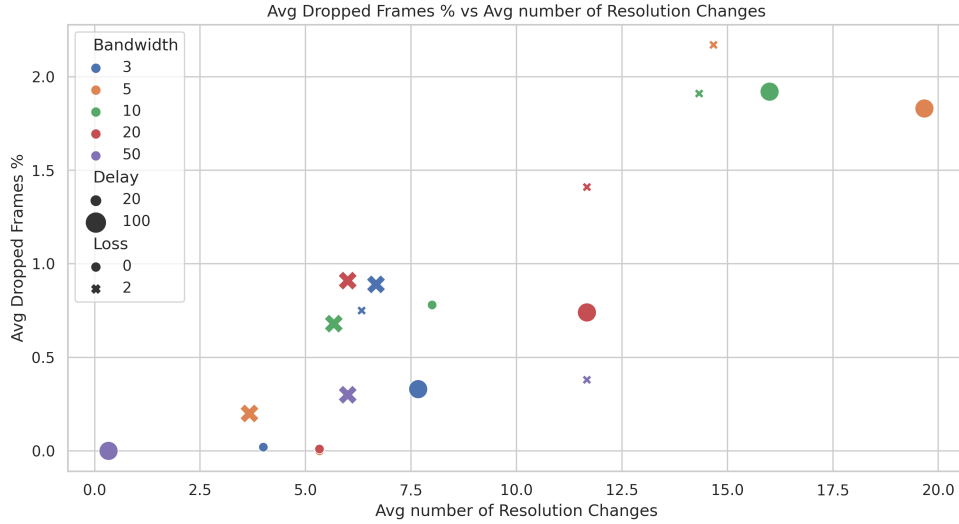


Figure 5.9: Average frames dropped percentage and resolution changes

and low loss seem to result in both high rebuffering events and resolution changes. See Figure 5.11.

5.3 Analysis of Mininet with Traffic

This chapter compares the metrics derived from Mininet testing with and without the presence of network traffic. Particular attention is paid to the scenarios with 5 Mbps and 50 Mbps bandwidth limitations.

5.3.1 Low Delay (Zero loss)

The most pronounced differences between traffic and no-traffic scenarios were observed in low-delay tests at both 5 Mbps and 50 Mbps. In the 50 Mbps case, data recorded with traffic stays within the two grey lines. The initially high download rate fills the buffer, followed by a buffer drop. In response, the client struggles to maintain its desired resolution due to conflicting bitrate demands. This causes a sudden, visually disruptive drop in resolution and bitrate, accompanied by dropped frames. The client then adjusts to a higher resolution based on network availability. When ample bandwidth is consistently present, resolution stabilizes quickly. However, limited initial bandwidth means this

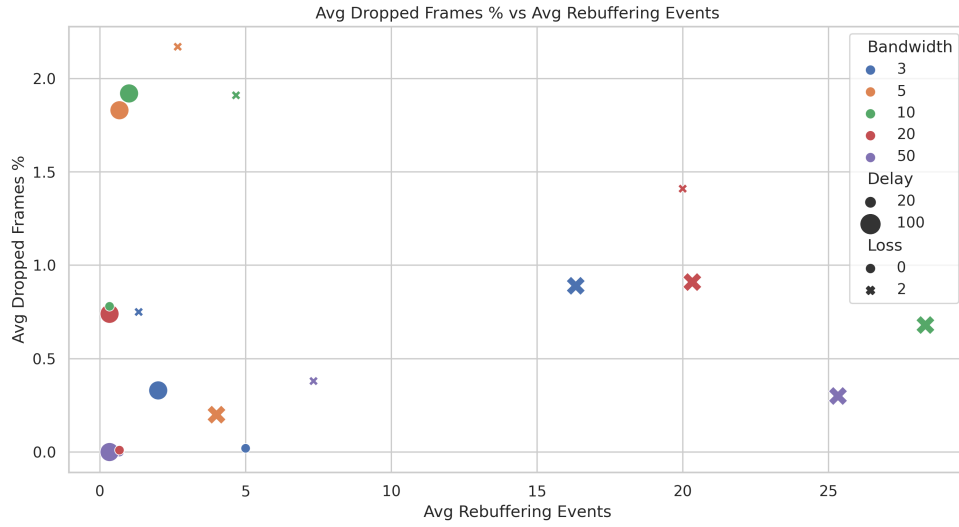


Figure 5.10: Average frames dropped percentage and rebuffering events

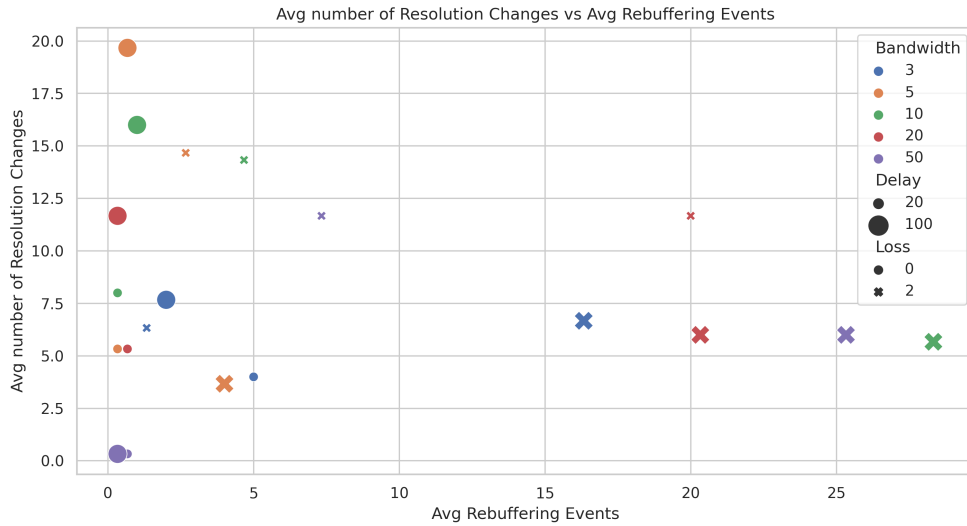


Figure 5.11: Average resolution changes and rebuffering events

stabilization process involves more fluctuation. Once traffic ends, the client optimizes bandwidth use. This often means transitioning to a higher bitrate and resolution and increasing the amount of video buffered as a safeguard against future disruptions. See Figures 5.12, 5.13, 5.14, and 5.15.

5.3.2 High Delay (Zero loss)

This scenario exhibits patterns like those seen in low-delay/no-loss tests. However, traffic competing for the bottleneck link creates further difficulties in maintaining stable resolution in both the 5 Mbps and 50 Mbps cases. This leads to more drastic resolution changes, negatively impacting viewing experience with continuous, noticeable quality shifts. Both cases experience more dropped frames, with a marginal increase in the 5 Mbps case compared to the no-traffic test. An interesting side effect in the 50 Mbps case was that, compared to the equivalent no-traffic, low-delay test, traffic caused the highest download rates (achieved during initial burst utilization) to be lower. See Figures 5.16, 5.17, 5.18, and 5.19.

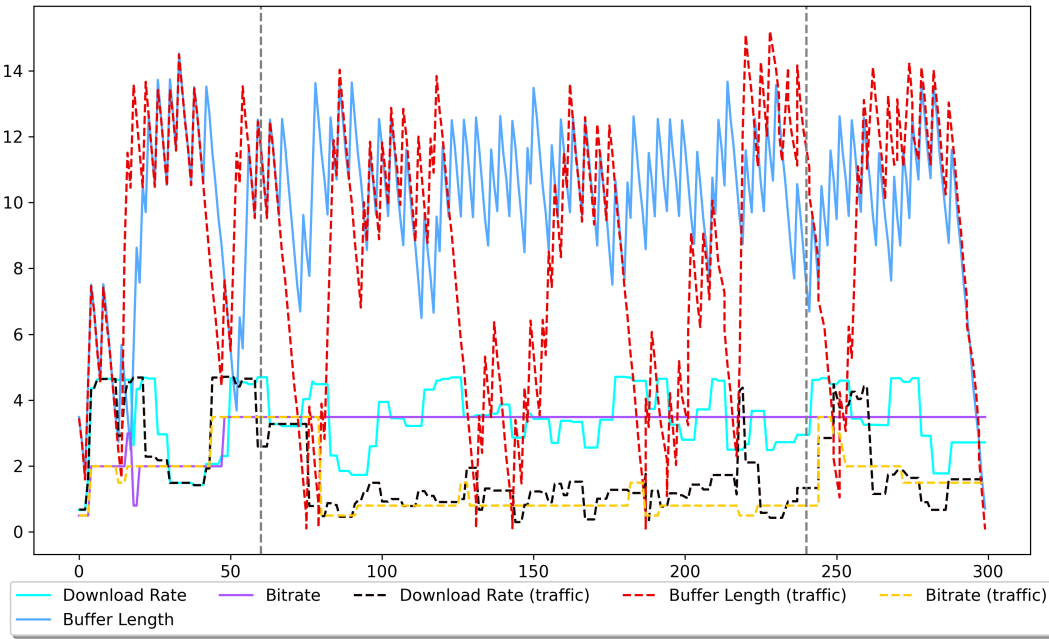


Figure 5.12: 5 Mbps low delay and Zero loss with traffic

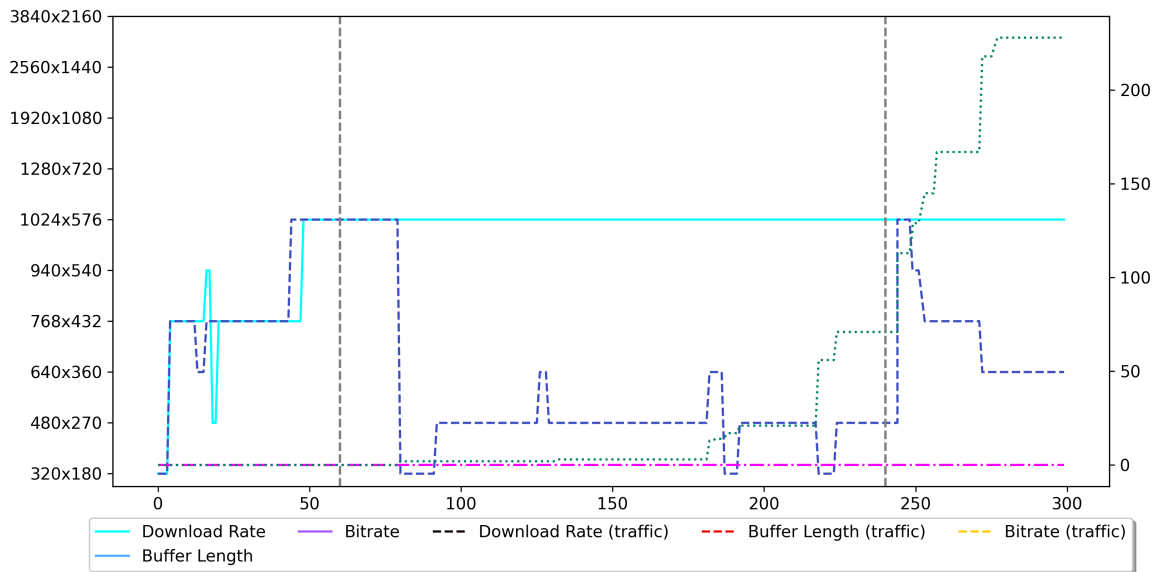


Figure 5.13: 5 Mbps low delay and Zero loss with traffic

5.3.3 Low Delay (low loss)

Data logged with traffic showed minimal deviation from no-traffic results. This indicates that existing network conditions were already a strong limiting factor on video streaming performance. As a result, introducing traffic caused only slightly lower bandwidth utilization. Lower download rates, less frequent resolution changes, and a minor increase in dropped frames were observed. See Figures 5.20, 5.21, 5.22, and 5.23.

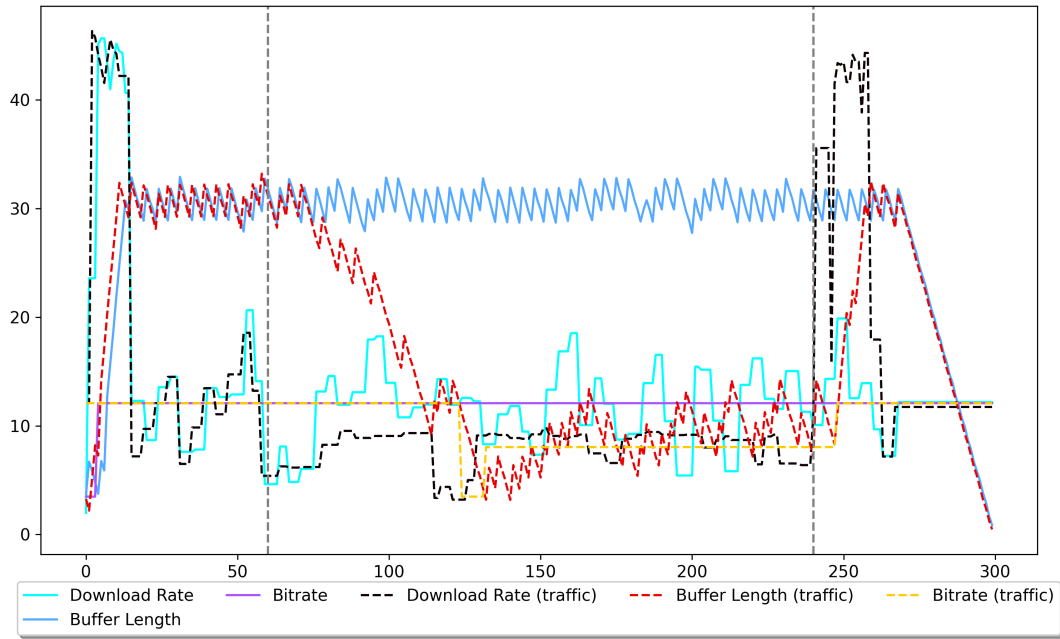


Figure 5.14: 50 Mbps low delay and Zero loss with traffic

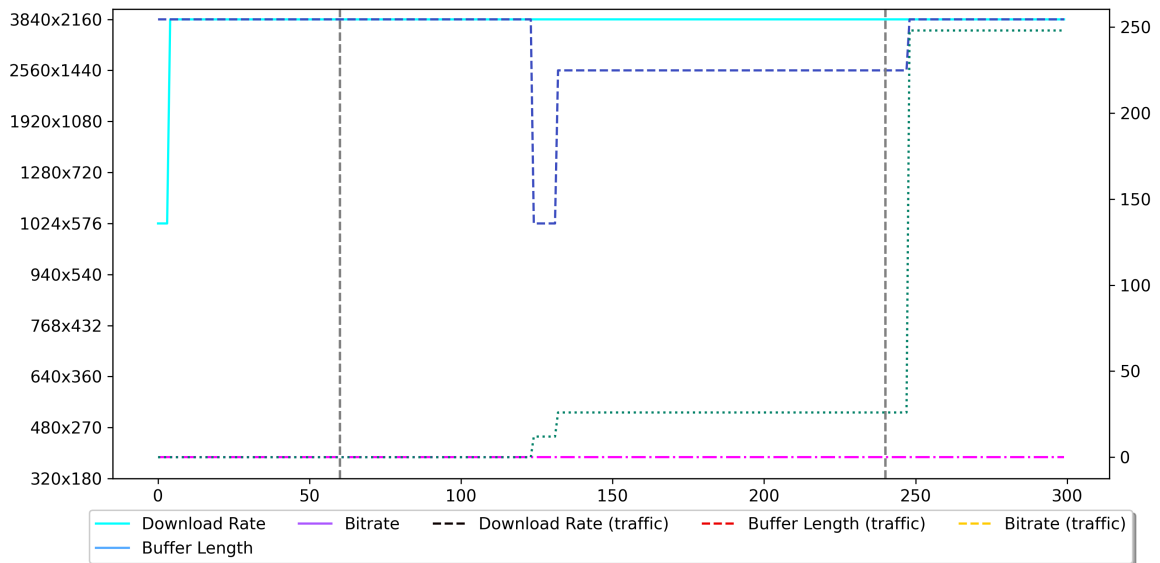


Figure 5.15: 50 Mbps low delay and Zero loss with traffic

5.3.4 High Delay (low loss)

Tests with or without traffic displayed minimal differences, highlighting the severe impact of the network conditions themselves on video streaming. Interestingly, in the 5 Mbps/traffic case, fewer frames were dropped compared to the no-traffic equivalent. A likely explanation is that fewer resolution spikes occurred, as the lowest resolution (180p) was consistently selected for much of the test, reducing the potential for dropped frames triggered by resolution changes. See Figures 5.24, 5.25, 5.26, and 5.27.

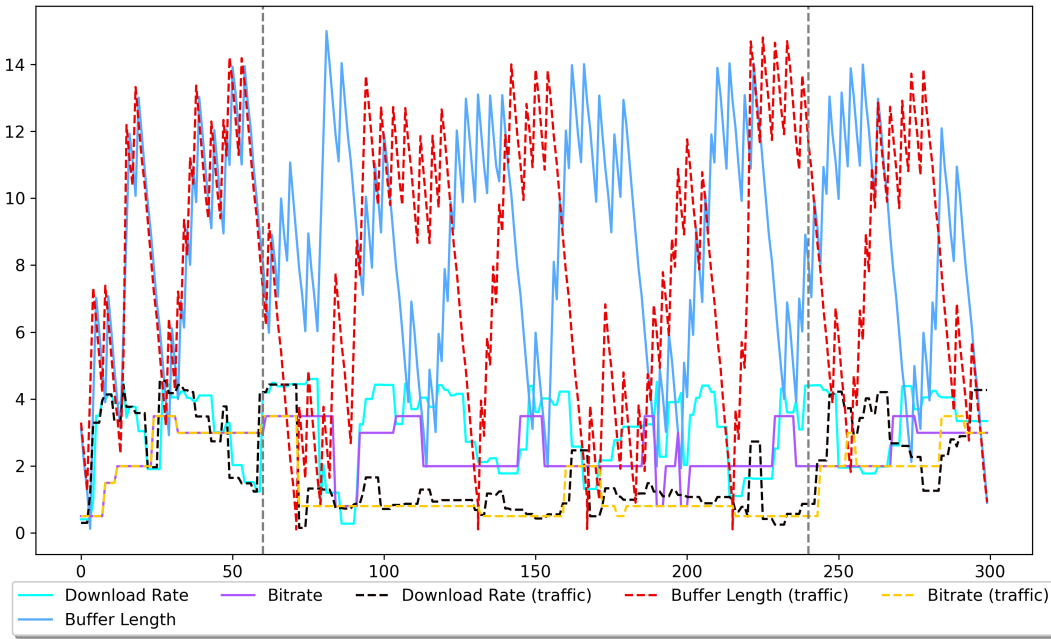


Figure 5.16: 5 Mbps high delay and Zero loss with traffic

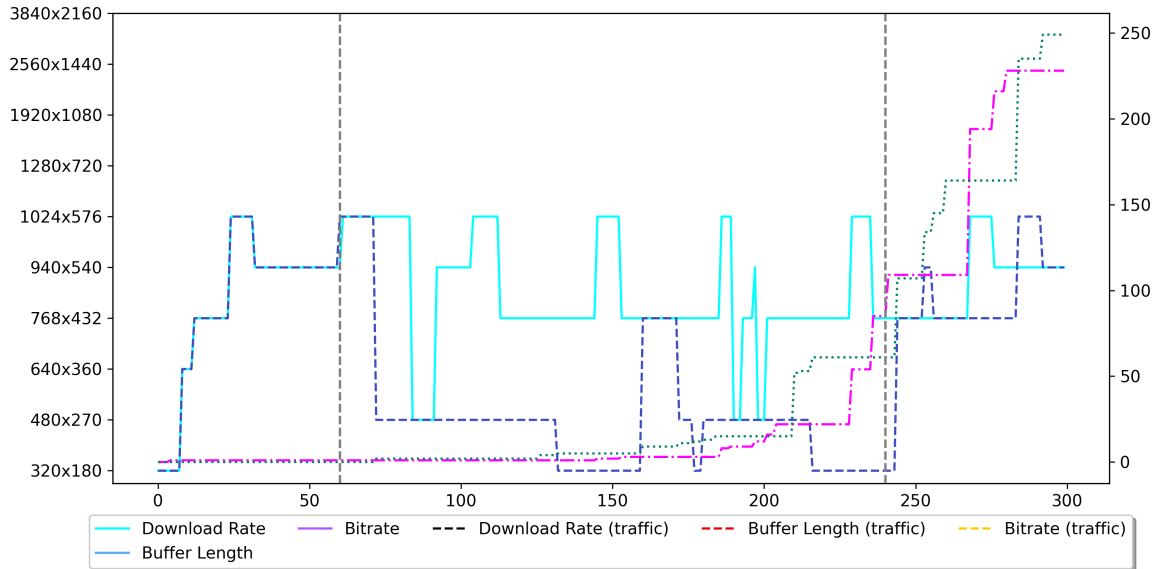


Figure 5.17: 5 Mbps high delay and Zero loss with traffic

5.4 Analysis of Impact Made by Geographical Distance

This analysis investigates the performance differences between tests conducted with a nearby university server and those conducted with a remote AWS server. The test point was approximately 3 km from the university server and 7000 km from the AWS server. As expected, this geographical distance significantly impacted network latency. The university server exhibited pings between 13 ms and 17 ms, while the AWS server had pings between 120 ms and 130 ms.

Despite the latency differences, both the university and AWS servers were able to stream video at maximum resolution without visible issues. The primary observable distinctions were in startup delay and segment timings:

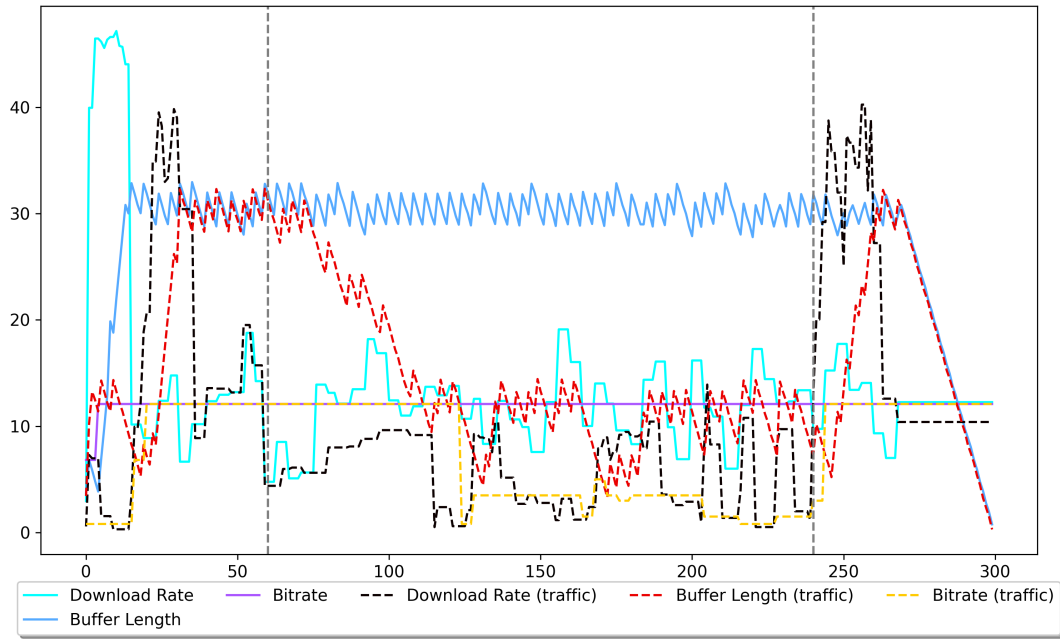


Figure 5.18: 50 Mbps high delay and Zero loss with traffic

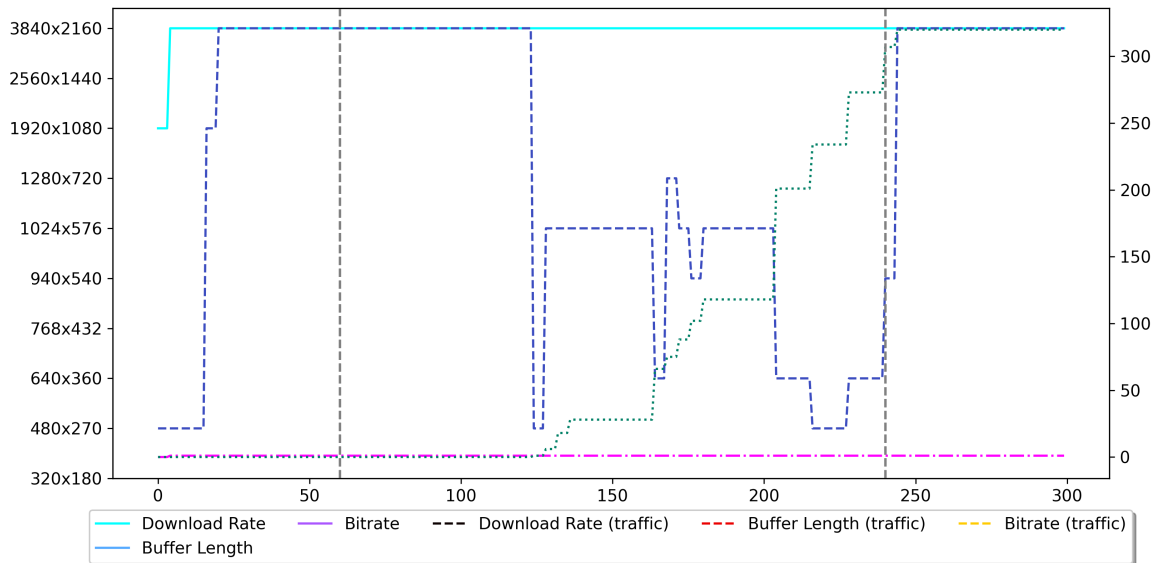


Figure 5.19: 50 Mbps high delay and Zero loss with traffic

- **Segment Response Times:** The university server averaged a 17.9 ms wait for the start of a segment response and 992 ms for the full response. The AWS server averaged a 225 ms wait for the start of a response and approximately 1.5 seconds for the full response. This reflects an approximate 728 ms difference in average waiting time between the two servers.
- **Startup Delay:** The AWS server demonstrated an average startup delay of approximately 1 second, compared to 344 ms for the university server. This represents a difference of approximately 650 ms.
- **Initial Download Burst:** The university server exhibited a peak download rate of 80 Mbps to fill

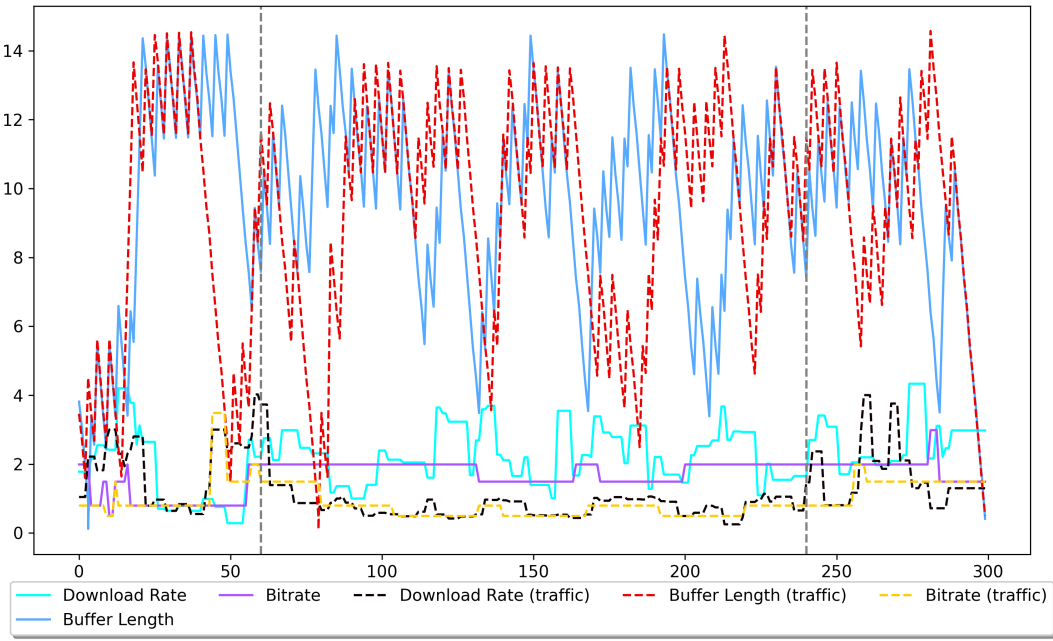


Figure 5.20: 5 Mbps low delay and low loss with traffic

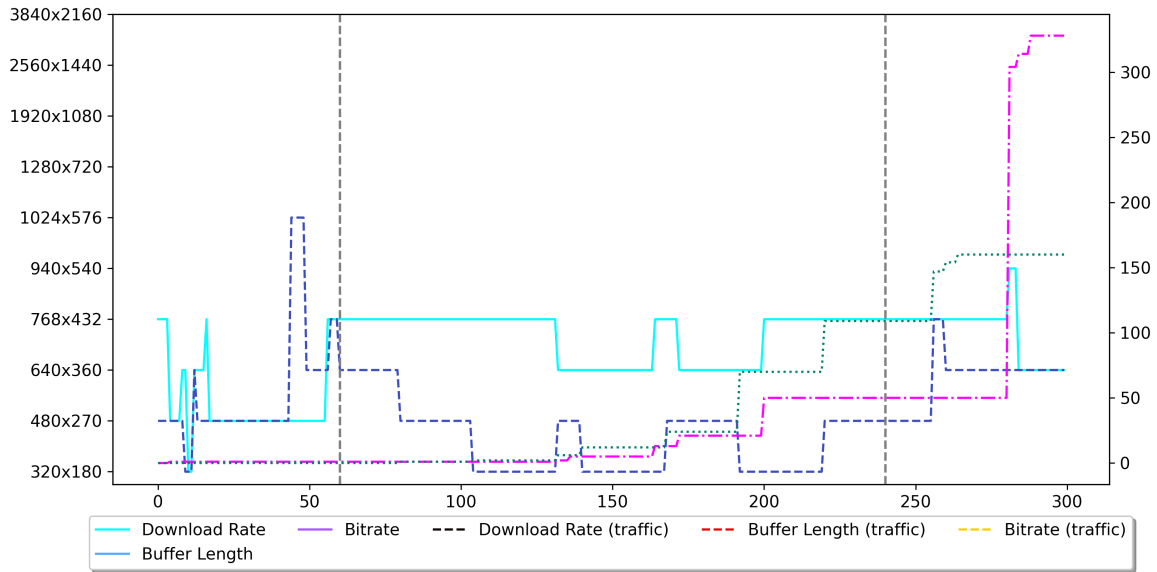


Figure 5.21: 5 Mbps low delay and low loss with traffic

the buffer, while the AWS server reached approximately 50 Mbps.

Importantly, these variations did not result in perceptible differences in the final video output. Both streams consistently maintained maximum resolution without any frame drops. See Figures 5.28, 5.29, 5.30, and 5.31.

5.5 Result and Future Research

The detailed analysis of MPEG-DASH streaming under various network conditions revealed several key insights. Network parameters like bandwidth, latency, and packet loss have a significant impact

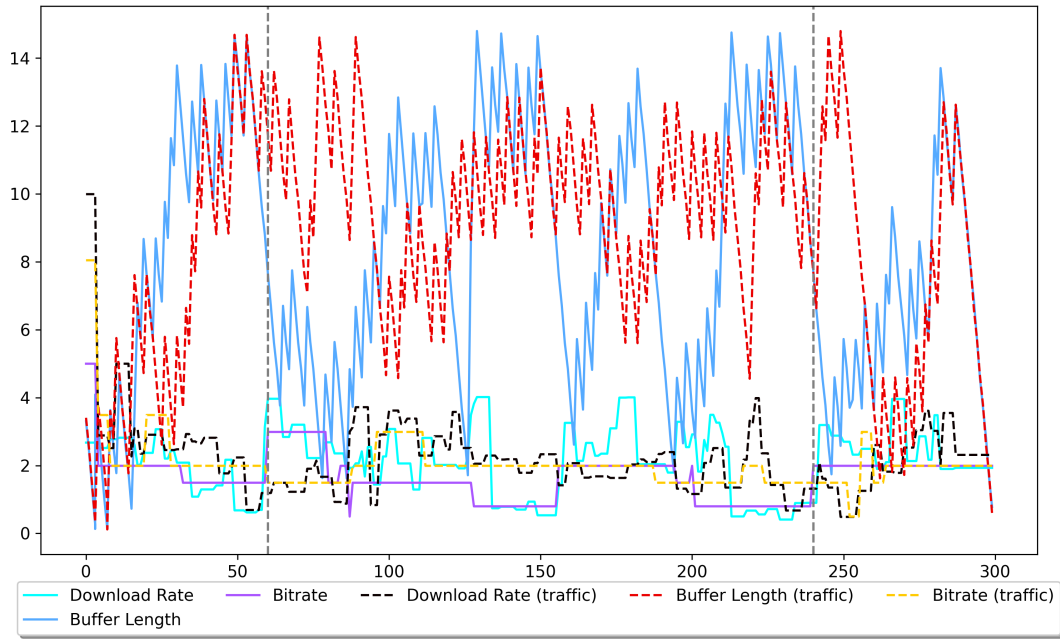


Figure 5.22: 50 Mbps low delay and low loss with traffic

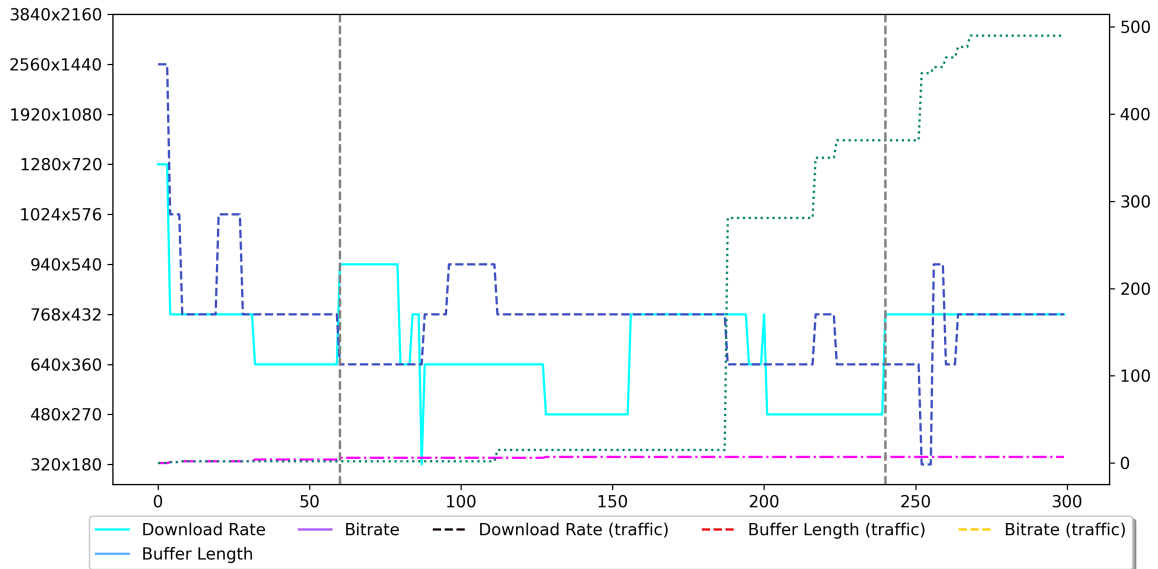


Figure 5.23: 50 Mbps low delay and low loss with traffic

on streaming performance. Increased bandwidth consistently led to improved performance with lower startup delays, more robust buffer health, and support for higher resolution streaming. Higher latency resulted in longer startup delays and more frequent resolution fluctuations as the client dynamically adapted. Packet loss negatively impacted streaming performance, especially when bandwidth was limited, causing buffer instability and lower resolutions. Additionally, background network traffic further compounded the challenges of maintaining stable resolution, leading to more noticeable quality shifts.

The DASH player employed different buffer management strategies depending on network conditions. It exhibited aggressive buffer filling in high-bandwidth, low-latency situations and was more

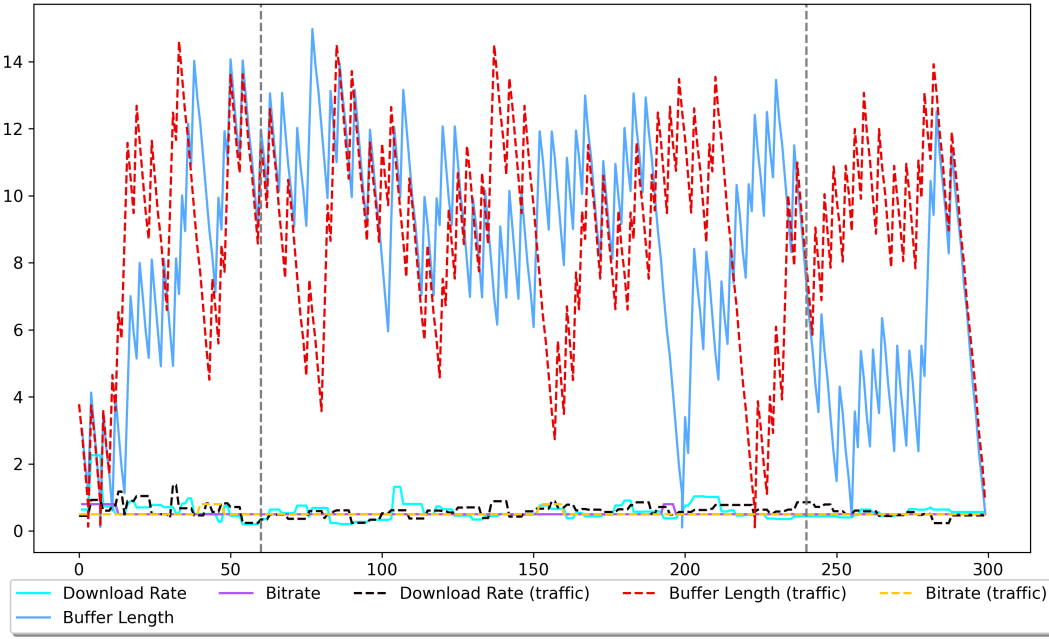


Figure 5.24: 5 Mbps high delay and low loss with traffic

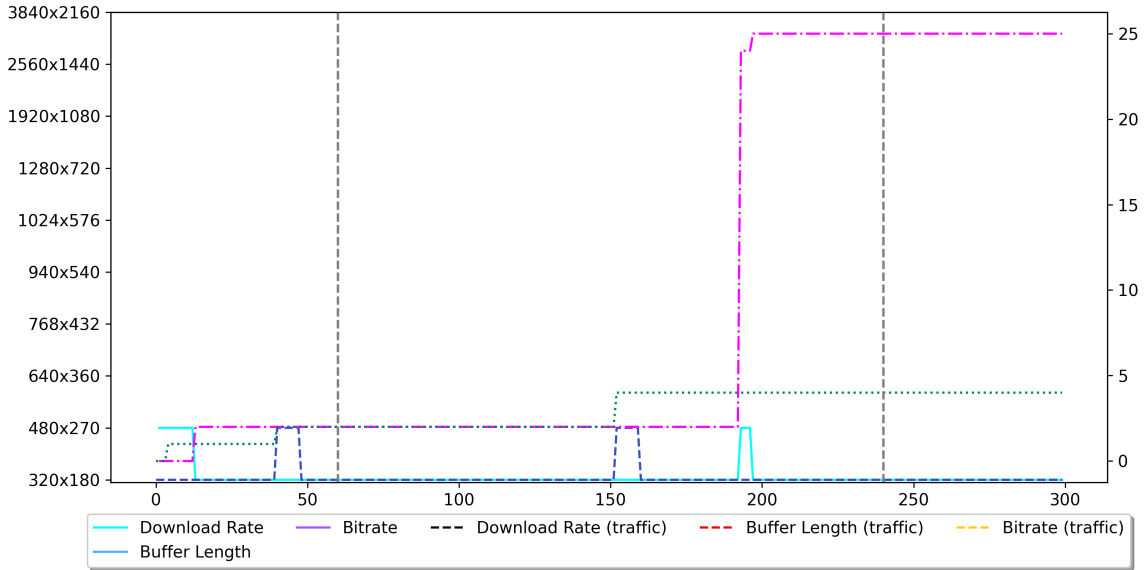


Figure 5.25: 5 Mbps high delay and low loss with traffic

conservative in bandwidth-limited scenarios. Clients attempted to maximize resolution whenever possible, with network stability playing a crucial role in resolution consistency. Unstable connections were a major contributor to frequent, disruptive quality shifts. Even in scenarios with both high latency and low packet loss, MPEG-DASH's adaptation mechanisms struggled to provide an optimal video streaming experience.

Our tests also analyzed the specific impact of geographical distance on latency. Increased latency, resulting from greater distances, led to longer startup delays and segment download times. Interestingly, this did not result in a substantial difference in final perceived video quality for the tests comparing a nearby university server with a geographically distant AWS server. This underscores

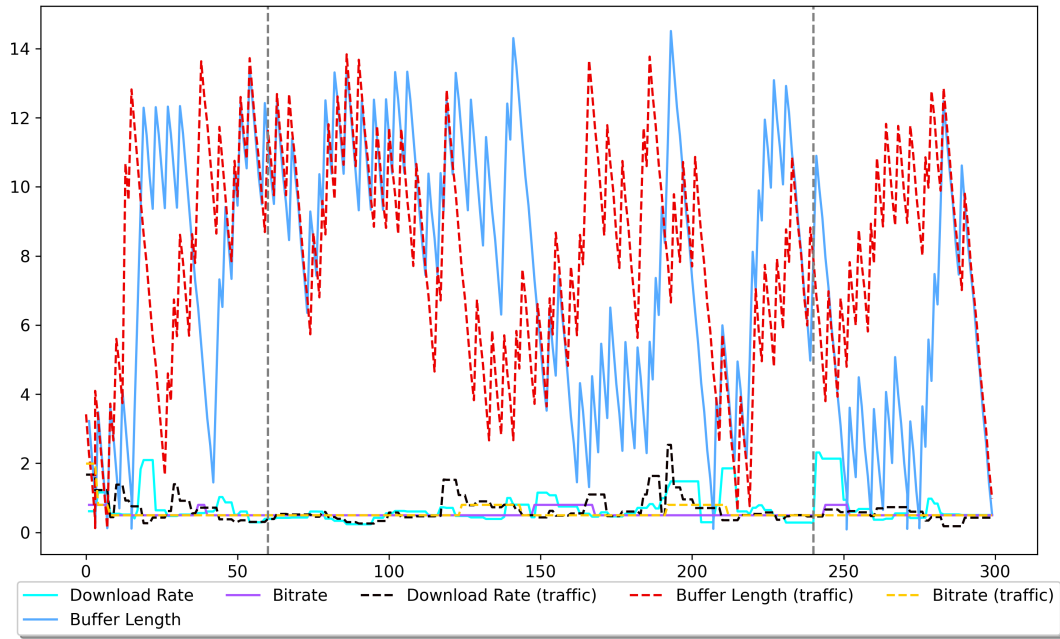


Figure 5.26: 50 Mbps high delay and low loss with traffic

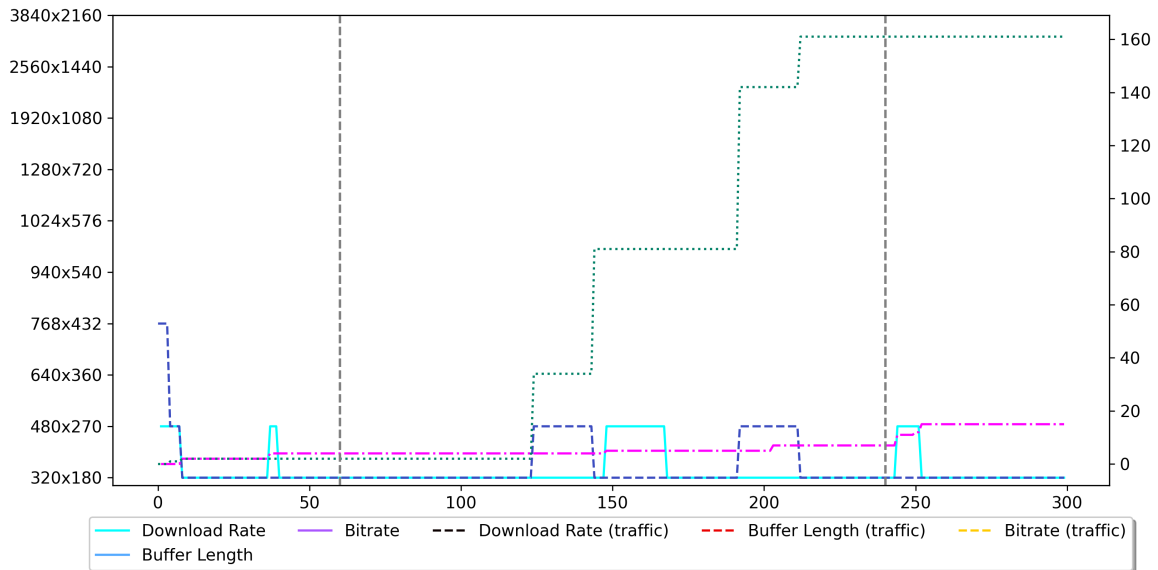


Figure 5.27: 50 Mbps high delay and low loss with traffic

the DASH player's ability to compensate for increased latency, delivering an acceptable streaming experience.

Finally, it is worth noting that part of our findings stem from controlled experiments. Performance under real-world network environments with additional variables may differ.

Future research areas could include conducting testing on mobile devices under cellular networks to evaluate other realistic usage scenarios, integrating Quality of Experience (QoE) measures to understand subjective user perception, and exploring ways to enhance bitrate selection algorithms to further optimize viewer experience in dynamic network situations.

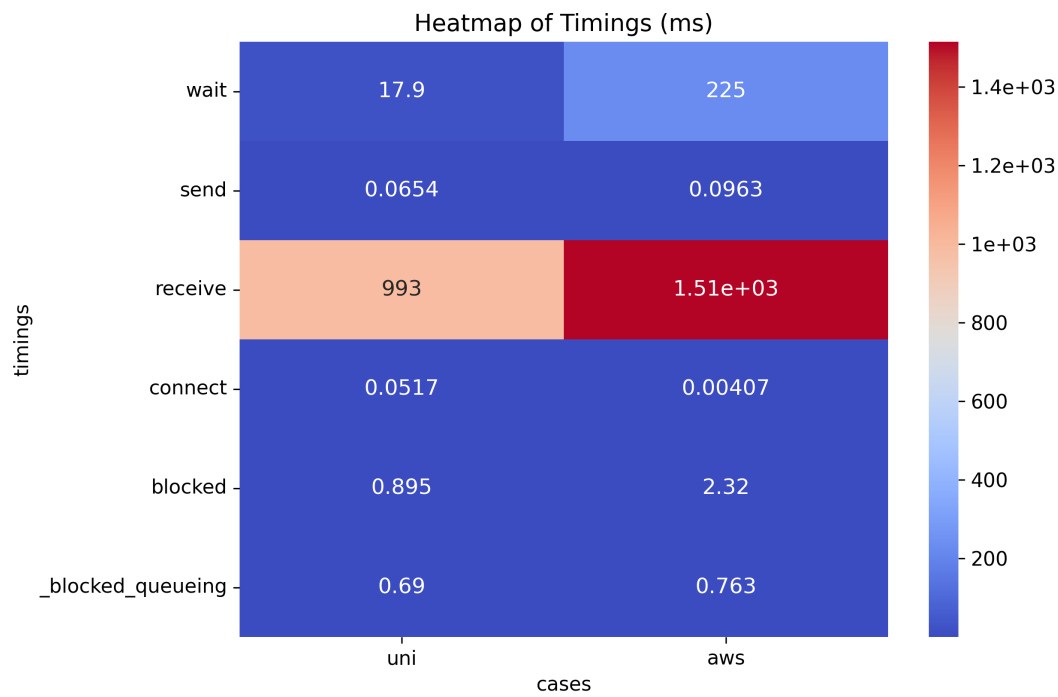


Figure 5.28: Timings

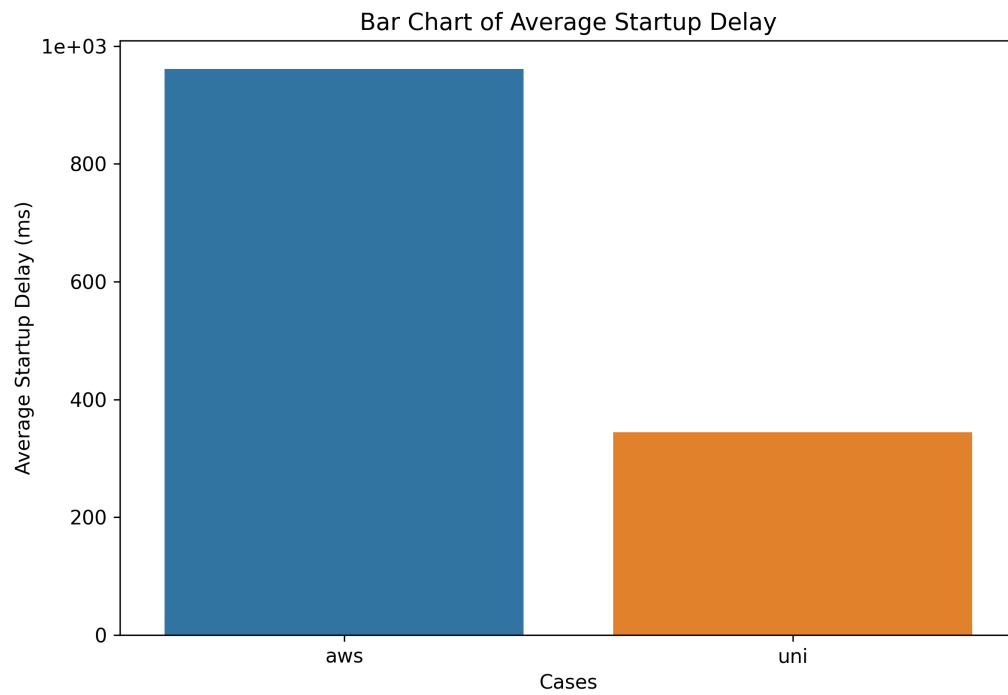


Figure 5.29: Average startup delay

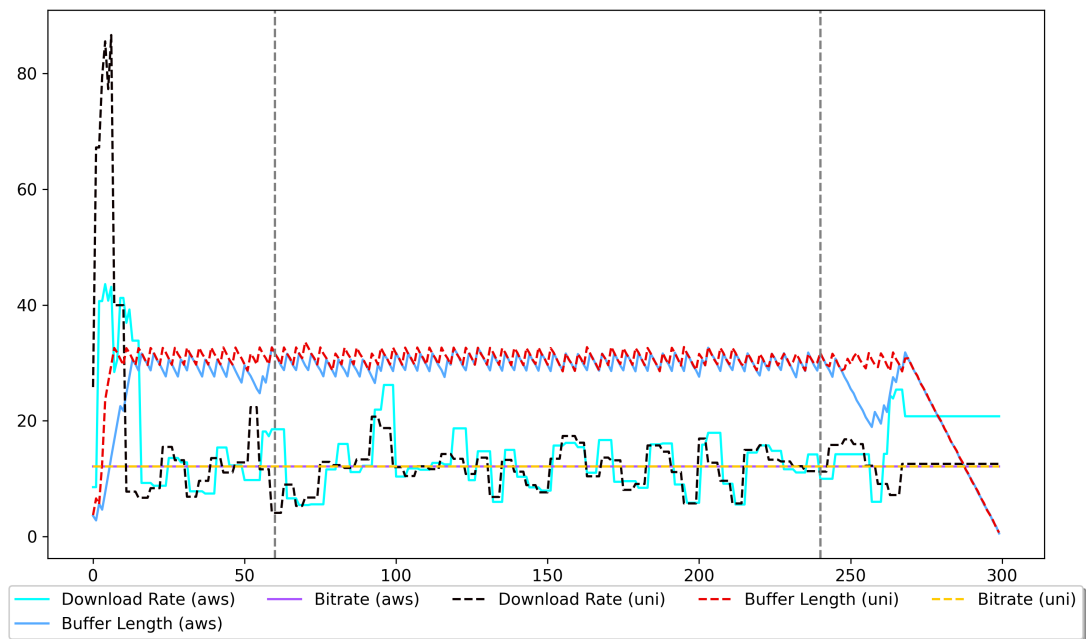


Figure 5.30:

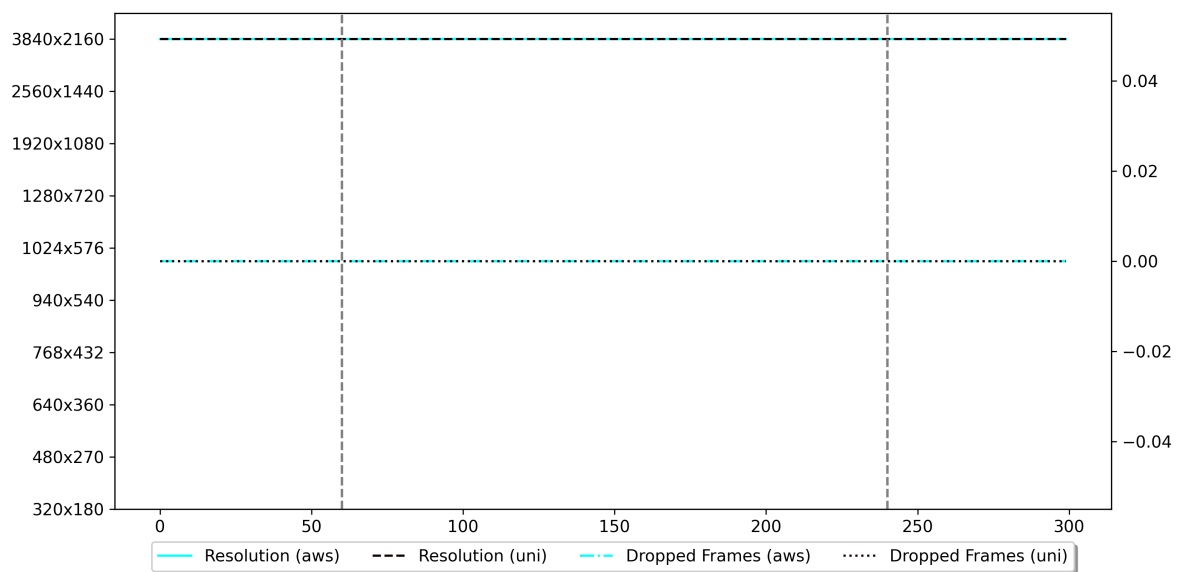


Figure 5.31:

Bibliography

- [1] A beginner's guide to mininet. <https://www.opensourceforu.com/2017/04/beginners-guide-mininet>. Accessed 18/02/2023.
- [2] Chapter 2. using the opendaylight user interface (dlux). https://nexus.opendaylight.org/content/sites/site/org.opendaylight.docs/master/userguide/manuals/userguide/bk-user-guide/content/_using_the_opendaylight_user_interface_dlux.html. Accessed 18/02/2023.
- [3] Chart.js tutorial. <https://www.geeksforgeeks.org/chart-js-tutorial/>. Accessed 18/02/2023.
- [4] Digital 2023: Global overview report. <https://datareportal.com/reports/digital-2023-global-overview-report>. Website and slide presentation, see slides 56, 57, 59 and 90, accessed 18/02/2023.
- [5] Download/get started with mininet. <http://mininet.org/download/>. Accessed 18/02/2023.
- [6] Dynamic adaptive streaming over http. https://it.wikipedia.org/wiki/Dynamic_Adaptive_Streaming_over_HTTP. Accessed 18/02/2023.
- [7] Every video format, codec, and container explained. <https://api.video/blog/product-updates/every-video-format-codec-and-container-explained/>. Accessed 18/02/2023.
- [8] Gpac project on advanced concept. https://en.wikipedia.org/wiki/GPAC_Project_on_Advanced_Content. Accessed 18/02/2023.
- [9] Gpac rearchitecture. <https://github.com/gpac/gpac/wiki/Rearchitecture>. Accessed 18/02/2023.
- [10] H.264. <https://trac.ffmpeg.org/wiki/Encode/H.264>. Accessed 18/02/2023.
- [11] Hls vs. dash. <https://www.wowza.com/blog/hls-vs-dash>. Accessed 21/02/2023.
- [12] Http live streaming. https://en.wikipedia.org/wiki/HTTP_Live_Streaming. Accessed 21/02/2023.
- [13] Mininet walkthrough. <http://mininet.org/walkthrough/>. Accessed 18/02/2023.
- [14] Mpeg-dash content generation with mp4box dash and x264. <https://bitmovin.com/mp4box-dash-content-generation-x264>. Accessed 18/02/2023.
- [15] Mpeg-dash (dynamic adaptive streaming over http). <https://bitmovin.com/dynamic-adaptive-streaming-http-mpeg-dash>. Accessed 18/02/2023.
- [16] A reference client implementation for the playback of mpeg dash via javascript and compliant browsers. <https://github.com/Dash-Industry-Forum/dash.js>. Accessed 18/02/2023.
- [17] Samples. <https://reference.dashif.org/dash.js/v4.4.0/samples/index.html>. Accessed 18/02/2023.
- [18] Thesis related code. <https://github.com/MaiDormo/thesis>. Accessed 18/02/2023.

- [19] Using the opendaylight user interface (dlux). [https://test-odl-docs.readthedocs.io/en/latest/user-guide/using-the-opendaylight-user-interface-\(dlux\).html](https://test-odl-docs.readthedocs.io/en/latest/user-guide/using-the-opendaylight-user-interface-(dlux).html). Accessed 18/02/2023.
- [20] Video codec. https://en.wikipedia.org/wiki/Video_codec. Accessed 18/02/2023.
- [21] What is mpeg-dash? and mpeg-dash vs. hls. <https://cloudinary.com/guides/video-formats/what-is-mpeg-dash-and-mpeg-dash-vs-hls>. Accessed 18/02/2023.
- [22] x264. <https://en.wikipedia.org/wiki/X264>. Accessed 18/02/2023.